SIEMENS

Publisher 1

COMOS PDMS Integration 2

COMOS 3D Viewing 3

References 4

COMOS

Lifecycle
3D Integration Administration

Operating Manual

11/2017 V 10.2.2
A5E37098309-AC

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

⚠ DANGER
indicates that death or severe personal injury **will** result if proper precautions are not taken.

⚠ WARNING
indicates that death or severe personal injury **may** result if proper precautions are not taken.

⚠ CAUTION
indicates that minor personal injury can result if proper precautions are not taken.

NOTICE
indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

⚠ WARNING
Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens AG. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens AG
Division Process Industries and Drives
Postfach 48 48
90026 NÜRNBERG
GERMANY

A5E37098309-AC
℗ 11/2017 Subject to change

Copyright © Siemens AG 2013 - 2017.
All rights reserved

Table of contents

1 Publisher....7

2 COMOS PDMS Integration....9

2.1 Software requirements....9

2.2 Installing COMOS PDMS Integration....10
2.2.1 Install....10
2.2.1.1 Local installation....10
2.2.1.2 Installing on a Citrix client....11
2.2.1.3 Installing on a Citrix client in silent mode....12
2.2.1.4 Installing on a Citrix server....13
2.2.1.5 Initialization file....14
2.2.1.6 Example text file for installation in silent mode....15
2.2.2 Specifying the path to the initialization file....16
2.2.3 Communication modes....16
2.2.4 Activating and deactivating the COMOS PDMS interface....17
2.2.5 UDAs....17

2.3 Setting up network mode....18
2.3.1 Creating configuration files....19
2.3.2 Editing the initialization file....20
2.3.3 "ComosPDMSInterfaceNetworkConfig.xml" file....20
2.3.4 Editing the network configuration file....22
2.3.5 Establishing network communication....23
2.3.6 Using the Server Directory Service....23
2.3.6.1 WCF....24
2.3.6.2 Installing the Server Directory Service....24
2.3.6.3 Starting the Server Directory Service....25
2.3.6.4 Setting the port and host of the SDS....25
2.3.6.5 Activating Portsharing....26
2.3.6.6 Enable use of the SDS for the engineering interface....27
2.3.6.7 Preparing the network configuration file for working with the SDS....27
2.3.6.8 Work environment variants....30
2.3.6.9 Working in a network....31
2.3.6.10 Working in multiple networks....39
2.3.6.11 Working via Citrix....40
2.3.6.12 WCF security options....44
2.3.6.13 Specifying WCF security options for the SDS....45
2.3.6.14 Specifying WCF security options for COMOS and PDM/E3D client....46
2.3.6.15 Specify the security settings for communication with the SDS via the service configuration file....47
2.3.6.16 Specify the security settings for communication between COMOS and PDMS/E3D via configuration file....50
2.3.6.17 Extending the configuration files for SDS....52
2.3.6.18 Using user groups....53
2.3.6.19 Using PDMS Engineering Interface with SDS....53

2.4 Configuring COMOS PDMS/E3D Engineering Interface....54

2.4.1 Entering COMOS PDMS/E3D Engineering Interface as an add-in....54
2.4.2 Restricting COMOS objects for the COMOS PDMS/E3D Engineering Interface....54
2.4.2.1 Creating queries....54
2.4.2.2 Assigning queries to users....55
2.4.3 Using classes and subclasses....55
2.4.4 Implement PML macro !!SAGPickUpQueryResult(!p is STRING)....56
2.4.5 Integrating custom icons for objects in the 3D workspace....56
2.4.6 Saving attributes of a PDMS/E3D model....57

2.5 Configuration basics for interface operations....57
2.5.1 Interface objects....57
2.5.2 Classes and subclasses....58
2.5.2.1 Class definition objects....59
2.5.2.2 Subclass definition objects of "Cables" and "Devices with labels" classes....60
2.5.2.3 Subclass definition objects of the "Queries" class....61
2.5.2.4 Subclass definition objects of the "Documents" class....61
2.5.2.5 "Folders for class/subclass definitions" object....62
2.5.3 Name mapping....63
2.5.3.1 Level rules of name mapping....64
2.5.3.2 Structural behavior....65
2.5.4 Working with area pipes....65
2.5.5 String Parameters....66
2.5.6 Restriction rules for owners....67
2.5.7 Model information....68
2.5.8 Pre/post executables....69
2.5.9 Character mapping....69
2.5.10 Units mapping....69
2.5.11 ZONE mapping and SITE mapping....70
2.5.12 Operation messages....70
2.5.13 Connection information....71

2.6 Configuring interface operations....71
2.6.1 Task-based administration....73
2.6.1.1 Performing the basic configuration....73
2.6.1.2 Configuring the use of COMOS queries in PDMS....75
2.6.1.3 Configuring the use of pre/post executables....75
2.6.1.4 Configuring the import of DocLinks....76
2.6.1.5 Configuring the import of AVEVA templates....76
2.6.2 Maintaining the standard table for PDMS element types....76
2.6.3 Standard tables for classes and subclasses....77
2.6.4 Creating the "Folders for class/subclass definitions" object....77
2.6.5 Configuring the "Folders for class/subclass definitions" object....78
2.6.5.1 Activating operation messages....78
2.6.5.2 Defining global variables....79
2.6.5.3 Using pre/post executables....79
2.6.5.4 Using character mapping....81
2.6.5.5 Using unit mapping....81
2.6.5.6 Using ZONE mapping and SITE mapping....82
2.6.6 Creating a class definition object....84
2.6.7 Creating subclass definition objects....84
2.6.8 Configure subclass definition objects of classes "Cable" and "Tagged item"....85
2.6.8.1 Defining PDMS element types....85
2.6.8.2 Defining structural behavior....86

2.6.8.3 Defining the base object for the creation of interface objects....87
2.6.8.4 Automatically inserting additional hierarchy levels....88
2.6.8.5 Specifying restriction rules for owners....89
2.6.8.6 Configuring the model....91
2.6.9 Configure 'MTO (PDMS)' class....92
2.6.10 Working with String Parameters in the name directory....93
2.6.10.1 Name directory....93
2.6.10.2 Tasks of the String parameter:....93
2.6.10.3 Structure of a String Parameter....94
2.6.10.4 Configuring String Parameters....95
2.6.10.5 Assigning a COMOS value to a variable....95
2.6.10.6 Assigning a PDMS value to a variable....96
2.6.10.7 Writing a PDMS value to a COMOS attribute (import)....96
2.6.10.8 Writing a COMOS value to a PDMS object (export)....97
2.6.10.9 Generating a component of the name mapping....97
2.6.10.10 Configuring a value comparison....98
2.6.10.11 Checking an owner....98
2.6.10.12 Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns....99
2.6.11 Configuring site mapping and location mapping in name mapping....100
2.6.11.1 Structure of the tables for unit mapping and location mapping....100
2.6.11.2 Mapping for the COMOS PDMS Interface....102
2.6.11.3 Mapping for the COMOS PDMS Engineering Interface....109
2.6.12 Creating and configuring query subclasses....115
2.6.12.1 Workflow....115
2.6.12.2 Creating and configuring query subclasses....116
2.6.12.3 Creating class and subclass definitions for queries....116
2.6.12.4 Configuring a subclass definition object for queries....117
2.6.13 Configuring "Import DocLinks"....118
2.6.13.1 Defining the entry for the target document in PDMS....118
2.6.13.2 Configuring "Import DocLinks" operation in COMOS....119
2.6.13.3 Defining the String Parameter for the document name....119
2.6.13.4 Configuring general settings....119
2.6.13.5 Defining owner restriction rules for draft objects....120
2.6.14 Synchronizing classes and subclasses in COMOS and PDMS....120
2.6.15 Importing classes, subclasses and "Folders for classes/subclasses" object from XML file....121
2.6.16 Importing AVEVA design templates....122
2.6.17 Configuring COMOS interface objects....123
2.6.17.1 Assigning the class, subclass, and PDMS element type....123
2.6.17.2 Overwriting the inherited model information....124

3 COMOS 3D Viewing....127

4 References....129

4.1 Category "Process engineering > PDMS/E3D Engineering Interface"....129

4.2 COMOS PDMS Integration....129
4.2.1 Structure of the standard table for PDMS element types....129
4.2.2 Structure of the standard table for classes....130
4.2.3 Structure of the standard table for subclasses....130
4.2.4 Attributes of the "Folders for class/subclass definitions" object....131
4.2.4.1 "General" tab....131
4.2.4.2 "Name directory" tab....132

4.2.4.3 "Pre/post executables" tab....132
4.2.4.4 "Character mapping" tab....132
4.2.4.5 "Unit mapping" tab....133
4.2.4.6 "Site mapping" and "Zone mapping" tabs....133
4.2.5 Attributes of the subclass definition objects of classes "Cable" and "Tagged item"....133
4.2.5.1 "General" tab....133
4.2.5.2 "Name mapping" tab....134
4.2.5.3 "Owner restriction rules" tab....135
4.2.5.4 "Model" tab....135
4.2.5.5 "Name directory" tab....136
4.2.6 Attributes of the subclass definition objects of the "Query" class....136
4.2.6.1 The "General" tab....136
4.2.7 Attributes of the subclass definition objects of the "Document" class....137
4.2.7.1 "General" tab....137
4.2.7.2 "Owner restriction rules" tab....138
4.2.7.3 "Name directory" tab....138
4.2.8 Attributes of the interface objects....138
4.2.8.1 Attributes of the "External 3D Interface" tab....138
4.2.9 Attributes of the design template objects....140
4.2.9.1 Attributes of the "External 3D Interface" tab....140

Publisher

1

AVEVA

PDMS™ and Everything3D™ (E3D) are software products from VEVA and are hereinafter referred to simply as PDMS or E3D.

Bentley

OpenPlant Modeler is a software product from Bentley® and is hereinafter referred to simply as OpenPlant Modeler.

COMOS PDMS Integration 2

2.1 Software requirements

COMOS PDMS/E3D Engineering Interface

General requirements

- PDMS 12.0 and higher
- E3D 1.1 or 2.1

The methods for installing COMOS PDMS Integration involve file-based or network-based communication. See also chapter Communication modes (Page 16). Different software requirements apply depending on which type of communication is used:

- File-based communication
  Citrix: up to and including 4.5
- Network-based communication
  Citrix: up to and including 7.6

PDMS pipe spec export

PDMS 12.0 and higher

Specification import to COMOS (PDMS/E3D PipeSpec Interface)

- PDMS 12.0 and higher
- E3D 1.1.0

COMOS PDMS Interface

- PDMS 12.0 and higher
- Citrix XenApp versions up to and including 4.5

2.2 Installing COMOS PDMS Integration

2.2.1 Install

COMOS PDMS Integration consists of the following interfaces:

- COMOS PDMS Interface
- COMOS PDMS/E3D Engineering Interface

All interfaces are installed together with the COMOS PDMS Integration product, .

The following installation modes are available:

- Local installation
- Installation on the Citrix server
- Installation on the Citrix client
- Installation on the Citrix client in silent mode

2.2.1.1 Local installation

Requirement

COMOS and PDMS are already installed.

Procedure

1. Insert the installation CD and start "setup.exe".
2. Select the "Local" installation mode:
3. Click "Next".
4. Select the installation directory.
   The following will be saved here:
   - Program files of the "COMOS PDMS Interface"
   - Program files of the "COMOS PDMS/E3D Engineering Interface"
   - Initialization file
     See also chapter Initialization file (Page 14).

5. Specify the PDMS directories:
   You can change the default settings of the PDMS directories:
   - "PML-Lib":
     If the "Install PML-Lib" option is activated: The PML-Lib supplied with the installation CD is copied to this folder.
   - "Working directory":
     The path to the directory that will be used for data exchange. This directory can also be specified at a later time in COMOS in the project properties.
   - "Configuration directory":
     The path to the directory that is used to exchange command files. This directory can also be specified at a later time in COMOS in the project properties.
   - "Executable":
     The path to the folder where "TalkToComos.exe" is saved. The Windows environment variable PATH should point to this directory. If not, the directory is attached to the Windows environment variable PATH.
   - "Install PML-Lib" option: See above.
   - "Register PDMS components" option:
     If you do not select the option, you must edit the "DesignAddins.xml" file in the installation directory of PDMS:
     Insert the following XML node: `<string>Comos.PDMSInterface.ComosAddin</string>`
6. Click "Next".
7. Select the installation directory.
8. To start the installation, click "Install".
   During installation, entries are made in the registry and the PDMS-Lib is installed.
9. Click "Finish".

2.2.1.2 Installing on a Citrix client

Requirement

- PDMS is installed on the Citrix client.
- COMOS is installed on the Citrix server.

Procedure

1. Insert the installation CD and start "setup.exe".
2. Select the "Citrix client" installation mode.
3. Click "Next".

4. Select the installation directory.
   The following will be saved here:
   - Program files of the "COMOS PDMS Interface"
   - Program files of the "COMOS PDMS/E3D Engineering Interface"
   - Initialization file
     See also chapter Initialization file (Page 14).
5. Click "Next".
6. Specify the communication directory into which the COMOS components of the PDMS/E3D side are to be copied.
7. Click "Next".
8. Optional: Change the default settings of the PDMS directories on the client system:
   - "PML-Lib":
     If the "Install PML-Lib" option is activated: The PML-Lib supplied with the installation CD is copied to this folder.
   - "Working directory":
     The path to the directory that will be used for data exchange. You can change this directory at a later time in the initialization file.
   - "Configuration directory":
     The path to the directory that is used to exchange command files. This directory can also be specified at a later time in COMOS in the project properties.
   - "Executable":
     The path to the folder where "TalkToComos.exe" is saved. The Windows environment variable PATH should point to this directory. If not, the directory is attached to the Windows environment variable PATH.
   - "Install PML-Lib" option: See above.
   - "Register PDMS components" option:
     Registers the COMOS interface in PDMS.
     If you do not select the option, you must edit the "DesignAdmin.xml" file in the installation directory of PDMS.
     Insert the following XML node: `<string>Comos.PDMSInterface.ComosAddin</string>`
9. Click "Next".
10. To start the installation, click "Install".
    During installation, entries are made in the registry and the PDMS-Lib is installed.
11. Click "Finish".

2.2.1.3 Installing on a Citrix client in silent mode

Installation in silent mode runs in the background and is controlled by a text file in which the administrator has entered all of the relevant information in advance. You do not need to make any further entries during installation.

Requirement

PDMS is installed on the Citrix client.

Procedure

1. Create a new text file.
2. Save the file in a local directory.
3. Enter the following paths in the file:
   See also chapter Example text file for installation in silent mode (Page 15).
   - Line 1: <Path to installation directory (storage location for INI file)>;<0 or 1>
     0: Directory will not be created.
     1: If the directory does not exist, it will be created
   - Line 2: <Path to working directory (communication directory and replacement directory)>
   - Line 3: <Path to configuration directory (document path and configuration path)>
   - Line 4: <Path to PMLLib directory>;<0 or 1>
     0: The directory is not created; the PMLLib is not copied
     1: The directory is created; the PMLLib is copied
   - Line 5: <Path to PDMS installation directory>;<0 or 1>
     1: COMOS PDMS interface is entered in the PDMS `DesignAddins.xml` file.
     0: COMOS PDMS interface is not entered in the PDMS DesignAddins.xml file
     In this case, you must insert the entry manually. See also chapter Entering COMOS PDMS/E3D Engineering Interface as an add-in (Page 54).
4. Use the command line to call the setup with the path to the created text file as a parameter.

Result

The installation executes the same steps as the client installation. See also chapter Installing on a Citrix client (Page 11).

2.2.1.4 Installing on a Citrix server

Requirement

COMOS is already installed on the Citrix server.

Procedure

1. Insert the installation CD and start "setup.exe".
   The start dialog field of the InstallShield Wizard opens.
2. Click "Next".
3. Select the "Citrix server" installation mode.
4. Click "Next".

5. Select the installation directory.
   The following will be saved here:
   - Program files of the "COMOS PDMS Interface"
   - Program files of the "COMOS PDMS/E3D Engineering Interface"
   - Initialization file
     See also chapter Initialization file (Page 14).
6. Click "Next".
7. Optional: Change the default settings of the PDMS directories:
   - "Working directory" field:
     The path to the directory that will be used for data exchange. Must point to the same folder that is entered during the Client installation.
     Input required.
   - "Configuration directory" field:
     The path to the directory that is used to exchange command files. Must point to the same folder that is entered during the Client installation.
     Input required.
8. Click "Next".
9. Click "Install".
   During the installation entries are made in the registry.
10. Click "Finish".

2.2.1.5 Initialization file

To guarantee that COMOS and PDMS use the same configuration, the configuration is saved in an initialization file. This initialization file is created during the installation of COMOS PDMS Interface. COMOS and PDMS use this file for communication and data exchange during an interface operation.

The initialization file is in XML format and has the following structure:

```
<ComosPDMSInterfaceConfiguration>
     <CommunicationFolder path="<path>"/>
     <ExchangeFolder path="<path>"/>
     <SubclassesFile path="<path>"/>
     <TalkToComos path="<path>"/>
     <Logfile path="<path>"/>
     <DocumentFolder path="<path>"/>
     <MTOTransferConfigurationFile path="<path>"/>
     <StartupFunctionsFile path="<path>"/>
     <ClientCommandService path="<path>"/>
     <StartClientCommandService value="<Boolean>"/>
</ComosPDMSInterfaceConfiguration>
```

The following nodes are included:

Node	Description
`CommunicationFolder`	• Mandatory information • Directory containing the XML files for command exchange. Created automatically, if not already present.
`ExchangeFolder`	• Mandatory information • Directory containing the XML files that are used for data exchange. Created automatically, if not already present.
`SubclassesFile`	• Mandatory information • Path to the Subclasses.xml configuration file
`TalkToComos`	• Mandatory information • Path to talkToComos.exe
`Logfile`	• Mandatory information • Path to the log file
`DocumentFolder`	Path to the document directory If this path is not specified, you cannot export any documents from the draft module. Created automatically, if not already present.
`MTOTransferConfigurationFile`	Path to file MTOExportSettings.dat. The file name here is an example only; you can choose any file name. If this path is not specified, you cannot transfer any MTO data to COMOS. You can find additional information on this topic in the manual "Material Management", keyword "MTOExportSettings.dat".
`StartupFunctionsFile`	Path to the file containing the StartupFunctions
`ClientCommandService`	Path to ClientCommandService.exe
`StartClientCommandService`	Flag which indicates whether ClientCommandService.exe is to be executed when AVEVA PDMS starts Possible values: • `true` • `false`

See also

Specifying the path to the initialization file (Page 16)

Setting up network mode (Page 18)

2.2.1.6 Example text file for installation in silent mode

```
D:\PDMSInterface\;1
D:\PDMSInterface\Work\
D:\PDMSInterface\Configuration\
```

```
D:\PDMSInterface\PMLLib\;1
D:\AVEVA\Plant\PDMS12.0.SP6\;0
```

See also

Installing on a Citrix client in silent mode (Page 12)

2.2.2 Specifying the path to the initialization file

Specifying the path to the initialization file in COMOS

1. Select the "Process engineering > PDMS" category in the project properties.
2. Click the "..." button of the "Initialization file" field.
3. In the file explorer, select the initialization file that was created during installation.

Specifying the path to the initialization file in PDMS

1. Specify the file location for the initialization file in an environment variable in the evars.bat file of your PDMS installation.
2. Define the environment variable as follows:
   `set COMOSPDMSINTERFACECONFIGURATION=<path>/<name of ini file>`

See also

Initialization file (Page 14)

2.2.3 Communication modes

COMOS PDMS interface

Installation type	Communication type
Local installation	File-based communication and inter-process communication
Citrix client installation	File-based communication
Citrix server installation	File-based communication

COMOS PDMS/E3D Engineering Interface

Installation type	Communication type
Local installation	File-based communication
Citrix client installation	File-based communication

Installation type	Communication type
Citrix server installation	File-based communication
Citrix client installation and Citrix server installation in network mode	Network communication via TCP/IP

2.2.4 Activating and deactivating the COMOS PDMS interface

In COMOS, you activate and deactivate the interface on a project-specific basis.

You only need to activate the interface once for each project. The interface remains activated, even in subsequent sessions, until you deactivate it.

To work with COMOS PDMS Interface, you must activate the interface in PDMS every time you restart PDMS. You can find additional information on this topic in the "3D Integration Operation" manual, keyword "Activating COMOS PDMS Interface for PDMS".

Activating COMOS PDMS Integration

1. Start COMOS.
2. Open the project in which you will work.
3. Open the project properties and go to the "PDMS" category.
4. Activate the "Interface active" option.
   COMOS PDMS Interface is being activated and made available in the COMOS menu.

Deactivating COMOS PDMS Integration

1. Start COMOS.
2. Open the project for which you want to deactivate the interface.
3. Open the project properties and go to the "PDMS" category.
4. Deactivate the "Interface active" option.

2.2.5 UDAs

To be able to use the PDMS interface, UDAs must be created in PDMS that contain the following information about the COMOS object:

- Connection status
- Configuration information
- Connection information

List with necessary UDAs

The following UDAs must be available in PDMS:

- ":ComosUID"
  Contains the SystemUID of the corresponding COMOS object.
- ":ComosCRefNo"
  Corresponds to the RefNo of the PDMS object.
- ":ComosStatus"
  Saves the object status.
- ":ComosName"
  This is the name of the corresponding COMOS object.
- ":ComosBaseOb"
  Saves the SystemFullName of the base object of the corresponding COMOS object. During an "Export to Comos" operation, this base object is used to create the corresponding COMOS object.
- ":ComosSClass"
  The name of the subclass to which this object belongs.

Create the UDAs

Create the UDAs in a project-related dictionary database. If no appropriate database is available, create the database with the help of the "!!!FCreateDictDB" macro. The following parameters must be passed to this macro in the form of a string:

- "MDB": Name of the MDB
- "TEAM": Name of the TEAM
- "DictDB": Name of the to be created database
- "DBNumber": Database number (must be specified by the administrator)

This macro does not create an MDB, but a TEAM, insofar this is necessary. Once the database has been created, the macro sets this. If you do not create the database with the macro, set the specified dictionary database to currently active to subsequently create the UDAs in the Lexicon module.

Once you have switched to the Lexicon module, create the UDAs with the !!ITSetComosUDAs macro underneath the UWRL.

2.3 Setting up network mode

As of Citrix 6.0, there may be some long waiting with file-based communication between client and server. To avoid these wait times, you can set up network communication as an alternative.

You can use network communication for the COMOS PDMS/E3D engineering interface.

Note

Protecting configuration files from changes

Make sure that users cannot edit or delete the configuration files.

Requirements

- See chapter Software requirements (Page 9).
- COMOS PDMS Integration is installed on the Citrix client and Citrix Server.

2.3.1 Creating configuration files

Procedure

1. Open the following file in the command line:
   "<PDMS installation directory>\bin\PDMSInterfaceIniFileGeneration.exe"
2. Use the following call parameters:
   PDMSInterfaceIniFileGeneration.exe <installation directory> network <working directory> <configuration directory> <server address>

Example

PDMSInterfaceIniFileGeneration.exe C:\PDMSInterface\ network C:\PDMSInterface\Work\ C:\PDMSInterface\Config\ 127.0.0.1

Result

The following files are created.

- In the installation directory: "ComosPDMSInterfaceInitialization.xml"
  See also section Initialization file (Page 14).
- In configuration directory: "ComosPDMSInterfaceNetworkConfig.xml"

Edit the configuration files.

- See section Editing the initialization file (Page 20).
- See section Editing the network configuration file (Page 22).

2.3.2 Editing the initialization file

You have the following options for adapting the initialization file for network mode:

- You replace your existing initialization file with the newly created initialization file. See also chapter Creating configuration files (Page 19).
- You copy the lines required for network mode to your existing initialization file.

To copy the required lines to your initialization file, follow these steps:

Procedure

1. Open the "ComosPDMSInterfaceInitialization.xml" file in the PDMS/E3D installation directory.
2. Insert the following lines:
```
<NetworkConfigFile path="<PDMS installation directory>\Config
\ComosPDMSInterfaceNetworkConfig.xml" />
<UseNetworkCommunication value="True" />
```

Configuration parameters	Description
`UseNetworkCommunication`	"`True`": Network communication is used "`False`": Network communication is not used
`NetworkConfigFile`	Path to the configuration file "ComosPDMSInterfaceNetworkConfig.xml" that includes the details for setting up network communication and the user port mappings. This parameter can remain empty if `UseNetworkCommunication` has the value "`False`".

Note

Do not delete existing XML elements in the initialization file

The `CommunicationFolder`, `ExchangeFolder` and `SubclassesFile` directories will continue to be used by the interface in network mode as well. Do not delete this information.

See also

Editing the network configuration file (Page 22)

Initialization file (Page 14)

2.3.3 "ComosPDMSInterfaceNetworkConfig.xml" file

You configure the network connections in the "ComosPDMSInterfaceNetworkConfig.xml" file.

The file has the same structure on the server and client sides. Some attributes are interpreted differently.

```
<ComosPDMSInterfaceNetworkConfiguration>
```

```
  <!-- HostAddress is local address on server, remote address on
client -->
  <!-- ConnectionTimeout and PingInterval values are in milliseconds
-->
  <Connection HostAddress="127.0.0.1" ConnectionTimeout="3000"
PingInterval="30000" MaxPingsToDisconnect="4"/>
  <Users>
    <!-- Port is local port on server, remote port on client -->
    <!-- Example: -->
    <!-- <User ComosName="ComosUser1" PDMSName="PDMSUser1"
Port="12345"/> -->
  </Users>
</ComosPDMSInterfaceNetworkConfiguration>
```

Explanation

"Connection" node

- "HostAddress" attribute:
  On the client (PDMS/E3D): Local host or IP address of a network adapter of the client.
  On the server (COMOS): Local host or IP address of the network adapter to be used. If there is only one network adapter, the attribute on the server can remain empty.
- "ConnectionTimeout" attribute:
  Time interval in milliseconds in which the client tries to establish a connection to the server. Once this time has expired, the attempt to establish a connection is canceled.
- "PingInterval" attribute:
  The value specifies the time interval in seconds that passes between two pings.
- "MaxPingsToDisconnect" attribute:
  Maximum number of pings without answer after which the network connection is interrupted.
- Attribute "ConnectionType"
  Values:
  - "TCP": Network communication via TCP/IP
  - "WCF": Network communication via WCF

  If the attribute is missing, WCF is enabled by default.
- Attribute "PortSharingEnabled"
  - "False": A single port must be defined for each user. If you are using TCP/IP, you must select the value "False".
  - "True": Multiple users can share a port. Use "True" if you use the Server Directory Service.

"Users" node

This node must include a "User" subnode for each COMOS user and PDMS/E3D user who is to use the interface via the network. You can specify as many users as you like as long as you assign a separate network port to each user.

- "User" node: These entries assign each user of the interface a network port of the server via which network communication is to take place.

  Note

  Separate port for each user

  Assign a separate port for each user. A port may be assigned multiple times only if it is ensured that the assigned users will not work in parallel and be connected to the server simultaneously.

  Exception: If you work with the Server Directory Service, multiple users can share a port.

- "ComosName" attribute:
  User name of the user in COMOS
- "PDMSName" attribute:
  User name of the user in PDMS/E3D.
- "Port" attribute:
  The network port that is assigned to the user on the server.

2.3.4 Editing the network configuration file

Depending on the number of network adapters, you need one or more network configuration files for server and clients. If you have several network adapters, use the HostAddress to select at the server end which network adapter of the server is to be used.

Requirement

You have created the "ComosPDMSInterfaceNetworkConfig.xml" file. See also chapter Creating configuration files (Page 19).

Procedure

1. Open the "ComosPDMSInterfaceNetworkConfig.xml" file.
2. Optional: Change the HostAddress.
3. Create a "User" node for each user of the interface.
   See also chapter "ComosPDMSInterfaceNetworkConfig.xml" file (Page 20).
4. Enter the following in the "Connection" node:
   Type="TCP"
5. Store the file in the directory that you have specified in the following node in the initialization file:
   `<NetworkConfigFile>`

2.3.5 Establishing network communication

Requirement

You have performed the previously described steps:

- Creating configuration files (Page 19)
- Editing the initialization file (Page 20)
- Editing the network configuration file (Page 22)
- "ComosPDMSInterfaceNetworkConfig.xml" file (Page 20)

Procedure

To establish the connection, you have to first open the interface at the COMOS end.

2.3.6 Using the Server Directory Service

The "Server Directory Service" (SDS) is a service that manages communication between COMOS and PDMS/E3D and is based on WCF.

The SDS can coordinate the communication within a network, between several networks, or in a Citrix environment. You can install the SDS on a server or on a separate computer. You specify the IP address and the port for the SDS in the network communication files that COMOS and PDMS/E3D access.

All the COMOS instances register themselves on the SDS. Their physical location is irrelevant. They may be physically located on several computers or on the same computer.

When a user starts PDMS/E3D, the SDS checks which COMOS instances have registered themselves and are available for this user. All the COMOS instances with which the PDMS/E3D user may connect are offered for selection. When the user selects an instance, the SDS establishes the connection between the two applications.

Terminology

- Server Directory Service (SDS)
- Server and client
  The terms "server" and "client" are considered from two different points of view in the context of the Server Directory Service.
  - Physical point of view
    From a physical point of view, the term "server" means a computer in a network.
    Example: A Citrix server from which COMOS is started by several users. The started COMOS instances are the COMOS clients.
  - Logical point of view:
    In the case of communication between COMOS and PDMS/E3D via the "PDMS/E3D Engineering Interface", each COMOS instance is regarded as a server. PDMS/E3D is a client of COMOS.
    The term "Server" in "Server Directory Service" refers to the logical point of view.

2.3.6.1 WCF

Terminology: WCF = Windows Communication Foundation

WCF is a framework by Microsoft. Additional information is available on the Microsoft website.

If you are working in network mode with TCP/IP, data is transmitted unencrypted by default. In order to increase data security, use WCF. You can expand and configure the security concepts provided by WCF.

WCF enables the following:

- Authentication between client and server
- Encryption of the data transmission

2.3.6.2 Installing the Server Directory Service

Install the Server Directory Service on a computer that can be accessed from all the computers that are to communicate with each other. You can install the service independent of COMOS.

If you work in more than one network, install the SDS in the network in which the COMOS instances are opened.

Procedure

1. Start the COMOS installation wizard.
2. Select "Setup language: German" or "Setup language: English". Click "Next".
3. Read the note and click "Next".
4. Click "Next".
5. Mark the option "I accept the terms of the license agreement. I confirm that I have read and understood the security information." and click "Next".
6. Select the setup type "Install". Click "Next".

7. In the component overview under "COMOS Main" select the "Directory Service" checkbox. Click "Next".
8. When you are ready to start the installation, click "Install".

2.3.6.3 Starting the Server Directory Service

Requirement

You have installed SDS. See also chapter Installing the Server Directory Service (Page 24).

Procedure

1. Open "Control Panel > Administrative Tools > Services".
2. Search for "Comos.MessageSystem.ServerDirectoryService".
3. Open the properties.
4. In the "Start type" list, select the entry "Manual" or "Automatic".
   If you select "Automatic", the service starts automatically when you restart the computer.
5. Click the "Start" button.

2.3.6.4 Setting the port and host of the SDS

If you do not specify a host address or a port for the Server Directory Service, the following is used by default:

- Host address: Localhost
  The SDS can be accessed through all network adapters of the host system.
- Port: 8884

You have several options for specifying a different host address or a different port.

Option 1: Creating an exe.config fie

1. Create a file called "Comos.External.Communication.ServerDirectoryService.exe.config".
   The file has to be located parallel to the following file:
   "Comos.External.Communication.ServerDirectoryService.exe"
   See also Configuring .NET programs (https://msdn.microsoft.com/en-us/library/aa374182(v=vs.85).aspx).
2. Add the following content:
   Example:
```
<configuration>
   <appSettings>
     <add key="TcpHost" value="<IP address|localhost>"/>
     <add key="TcpPort" value="<Port number>"/>
</appSettings>
</configuration>
```

Option 2: Specifying start parameters

You can set the parameters for the host and port in the properties dialog of the service.

1. Open "Control Panel > Administrative Tools > Services".
2. Search for "Comos.MessageSystem.ServerDirectoryService".
3. Open the properties.
4. Enter the host and port in the "Start parameter" field of the "General" tab.
   Example:
   -tcphost localhost -tcpport 8000

Option 3: Editing the registry

You can specify the parameters for the host and port in the registry.

1. Under "HKEY_LOCAL_MACHINE", create a "SOFTWARE\INNOTEC\COMOS \ServerDirectoryService" node.
2. Create a TcpHost value of the type string.
3. Specify the host address.
4. Create a TcpPort value of the type dword.
5. Specify the port.

2.3.6.5 Activating Portsharing

Activate port sharing if several COMOS instances are to be started on the same computer together with the PDMS/E3D engineering interface and you do not want to specify a single port for each COMOS user or Windows user.

For example it makes sense to use port sharing if COMOS is started on a Citrix server.

Procedure

1. Enable the .NET WCF port sharing service on the computer on which COMOS is installed. See also Enabling .Net TCP Port Sharing Service (https://msdn.microsoft.com/en-us/library/ms733925(v=vs.110).aspx).
2. By default, only system and network accounts have access to the .NET WCF port sharing service. However, if you use the SDS, all the COMOS users must have access. Assign the access rights.

Additional information about the configuration of the .NET WCF port sharing service:

- See also Configuring the Net.TCP Port Sharing Service (https://msdn.microsoft.com/en-us/library/aa702669(v=vs.110).aspx).
- See also Well-known SIDs (https://msdn.microsoft.com/en-us/library/windows/desktop/aa379649(v=vs.85).aspx).

Result

Port sharing is activated at the operating system end. You also need to activate it in the network configuration file that COMOS accesses. See also chapter Preparing the network configuration file for working with the SDS (Page 27).

2.3.6.6 Enable use of the SDS for the engineering interface

In order to use SDS when you open the COMOS PDMS Engineering Interface, you have to edit the configuration files.

Procedure

1. In the installation directory of the COMOS PDMS Engineering Interface, open the "ComosPDMSInterfaceConfiguration.xml" initialization file.
   The installation directory can always be found under the name "PDMSInterface".
2. Enter the following:
   <UseNetworkCommunication value="True"/>
   <NetworkConfigFile path="<path to the file "ComosPDMSInterfaceNetworkConfig.xml>"/>
   You can find the file in the installation directory of the COMOS PDMS Engineering Interface.
3. Ensure that all computers that are to communicate with each other via the SDS have access to the following files:
   – "ComosPDMSInterfaceConfiguration.xml"
   – "ComosPDMSInterfaceNetworkConfig.xml"
4. Ensure that the path to the following file is specified in the "Initialization file" field for the "PDMS/E3D Engineering Interface" category in the COMOS project properties: "ComosPDMSInterfaceConfiguration.xml"

2.3.6.7 Preparing the network configuration file for working with the SDS

Depending on how you structure the network configuration file, you use a common network configuration file or different network configuration files for servers (COMOS) and clients (PDMS/E3D).

If you use the SDS, the network configuration files have the following structure:

```
<ComosPDMSInterfaceNetworkConfiguration>
  <Connection HostAddress="localhost" Type="WCF" PortSharingEnabled="true"/>          ①
  <Users>
    <User ComosName="*" PDMSName="*" Port="1234"/>                                     ②
  </Users>
  <ServerDirectoryServices>
        <ServerDirectoryService Address="123.123.123.12:8884"/>                        ③
  </ServerDirectoryServices>
</ComosPDMSInterfaceNetworkConfiguration>
```

① "Connection" node

COMOS and PDMS/E3D use the specifications in this node in the following cases:

- If no SDS was configured
- If the SDS does not respond or is not configured correctly
- If no COMOS instance with which the PDMS/E3D can connect is found via the SDS
- If it is not possible to establish a connection to a COMOS instance that you have selected via the SDS

If one of these cases occurs, the connection between COMOS and PDMS/E3D is established without the support of SDS. The connection is then established via the specifications in the "Connection" and "Users" nodes.

Possible values for HostAddress:

- "localhost"
  - In the network configuration file of COMOS:
    COMOS is accessible for PDMS/E3D via all the network adapters of the computer on which it is installed.
  - In the network configuration file of PDMS/E3D:
    PDMS/E3D connects to a COMOS instance that is installed on the same computer and has been started.
- IP address
  - Effects in the network configuration file of COMOS:
    If you specify the IP address of a network adapter of the computer, COMOS can only be accessed via this network adapter by PDMS/E3D.
  - Effects in the network configuration file of PDMS/E3D:
    PDMS/E3D connects to a COMOS instance that can be accessed under the specified IP address.

Recommendation: Enter "localhost" as the HostAddress.

In order to use port sharing, the value in the attribute "PortSharingEnabled" has to have the value "True".

② "Users" node

- If you use port sharing, the attributes "ComosName" and "PDMSName" must have the placeholder "*" as their value.
- "Port" attribute:
  The value determines the port through which PDMS/E3D and COMOS connect. These specifications have to match in the network configuration file for COMOS and PDMS/E3D. This value is used both when you use the SDS and when communication via the SDS does not work and the specifications in the "Connection" node are used instead.

③ "ServerDirectoryServices" node

You can create a child node "ServerDirectoryService" for each SDS that is to be used. As a rule, you use only one SDS.

Possible values of the "Address" attribute:

- "localhost:8884"
  - Effects in the network configuration file for COMOS:
    COMOS instances log on at the SDS that is installed locally.
  - Effects in the network configuration file for PDMS/E3D:
    PDMS/E3D asks for available COMOS instances at the locally installed SDS. As a rule, the SDS is not installed on the same computer as PDMS/E3D.
- IP address
  - In the network configuration file for COMOS:
    Specify an IP address if you have assigned a specific network adapter of the computer to the SDS or if the SDS is installed on a different computer than COMOS.
  - In the network configuration file for PDMS/E3D:
    As a rule, the SDS is installed on a different computer than PDMS/E3D. You then specify the IP address of the computer on which the SDS is installed.

Requirement

You have carried out the pre-settings in the network configuration file. See also chapter Enable use of the SDS for the engineering interface (Page 27).

Procedure

1. Open the network communication file "ComosPDMSInterfaceNetworkConfig.xml".
2. Optional: In order to use port sharing add the following attribute to the <Connection> node: PortSharingEnabled="true"
   Activate the port sharing service. See also Chapter Activating Portsharing (Page 26).
3. Optional: If communication between PDMS/E3D and COMOS is to take place via TCP/IP, add the following attribute to the <Connection> node: Type="TCP"
   In this case, the network configuration files have to be configured in accordance with the default network communication. A separate port is assigned under each user. See also chapter "ComosPDMSInterfaceNetworkConfig.xml" file (Page 20).
   Possible values for "Type":
   - "TCP"
   - "WCF"

   If the "Type" attribute does not exist, communication takes place via WCF by default.

4. Enter one of the following values in the "HostAddress" attribute:
   - "localhost"
   - IP address of a network adapter or of a different computer
5. Enter the IP address and port of the SDS so that PDMS/E3D can query for available COMOS instances at the SDS.
```
<ServerDirectoryServices>
   <ServerDirectoryService Address="<Localhost or IP address:
8884>"/>
</ServerDirectoryServices>
```
6. Enter the following in the <User> node to enable communication regardless of the user name:
   - PDMSName="*"
   - ComosName="*"
7. Enter the port through which COMOS and PDMS/E3D are to communicate with each other in the <User> node.

Example

```
<ComosPDMSInterfaceNetworkConfiguration>

   <Connection HostAddress="localhost" Type"WCF"
PortSharingEnabled="true"/>

   <Users>
       <User Port="2700" PDMSName="*" ComosName="*"/>

   </Users>


   <ServerDirectoryServices>

       <ServerDirectoryService Address="localhost:8884"/>

   </ServerDirectoryServices>


</ComosPDMSInterfaceNetworkConfiguration>
```

2.3.6.8 Work environment variants

You can enable communication via the SDS in different working environment variants:

- In one network
  See also chapter Working in a network (Page 31).
- In multiple networks
  See also chapter Working in multiple networks (Page 39).
- Via Citrix
  See also chapter Working via Citrix (Page 40).

Depending on the work environment variant, PDMS/E3D users have access to COMOS instances started by them or to instances started by other users. You can define detailed access in the network configuration file and the WCF security options.

In all the following examples, it is assumed that you have not changed the default values for host and port. See also chapter Setting the port and host of the SDS (Page 25).

2.3.6.9 Working in a network

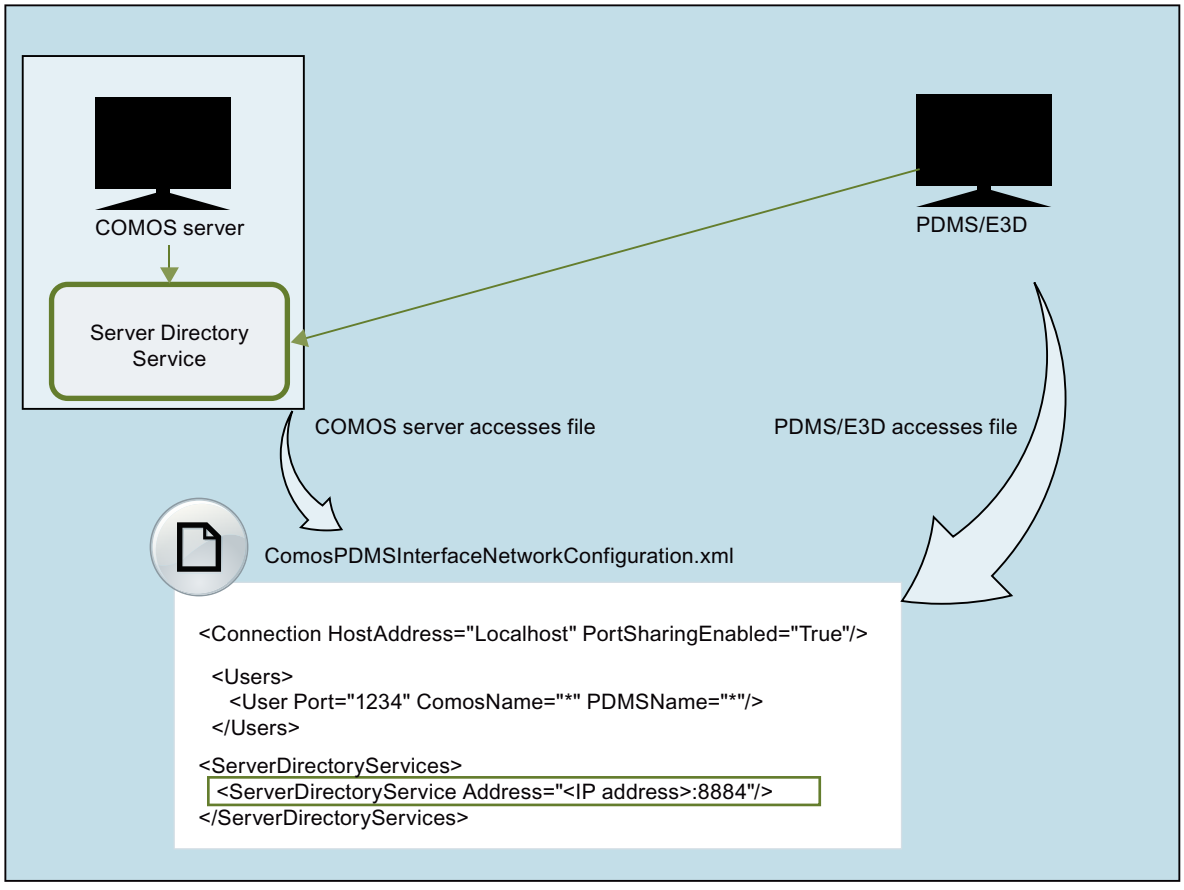
Example 1: A COMOS server

With this simplified representation, the SDS is installed on a server on which COMOS is installed. PDMS/E3D is installed in the same network on another computer.

The "ComosPDMSInterfaceNetworkConfig.xml" network configuration file that is used by both applications is located in a shared directory.

The file includes the following specifications:

- Host address="localhost"
  If the SDS cannot establish a connection between a COMOS client and PDMS/E3D, PDMS/E3D connects to a locally installed COMOS. This COMOS is not shown in the following graphic.
- Address="<IP address of the computer on which the SDS is installed>:8884"
  - All COMOS clients of the COMOS server log on at the SDS via this address.
  - All the PDMS/E3D instances query for free COMOS instances via this address at the SDS.



If the Engineering Interface is started in COMOS and PDMS/E3D, both applications connect with the SDS. The COMOS instance is offered for selection to the PDMS/E3D user. Once selected by the user, the SDS makes the connection between COMOS and PDMS/E3D.

If several COMOS instances exist, all of them are offered to the PDMS/E3D user for selection.

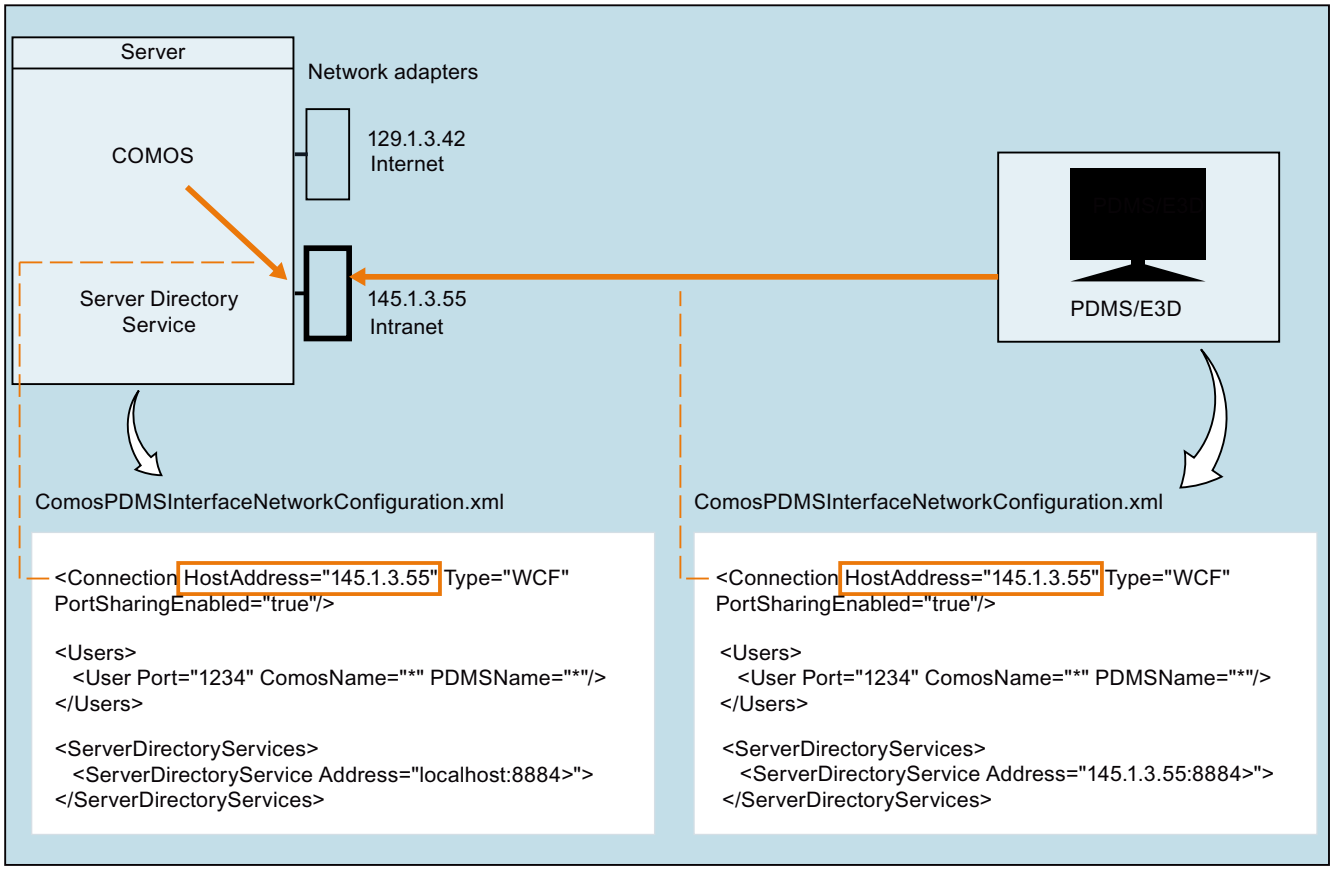
Example 2: Server with multiple network adapters

If you have multiple network adapters on a server and a COMOS server or SDS should only be accessible via one network adapter, specify the network adapter based on its IP address.

In this example, one network adapter exists for the Intranet and one for the Internet. COMOS and PDMS/E3D are to communicate via the Intranet.

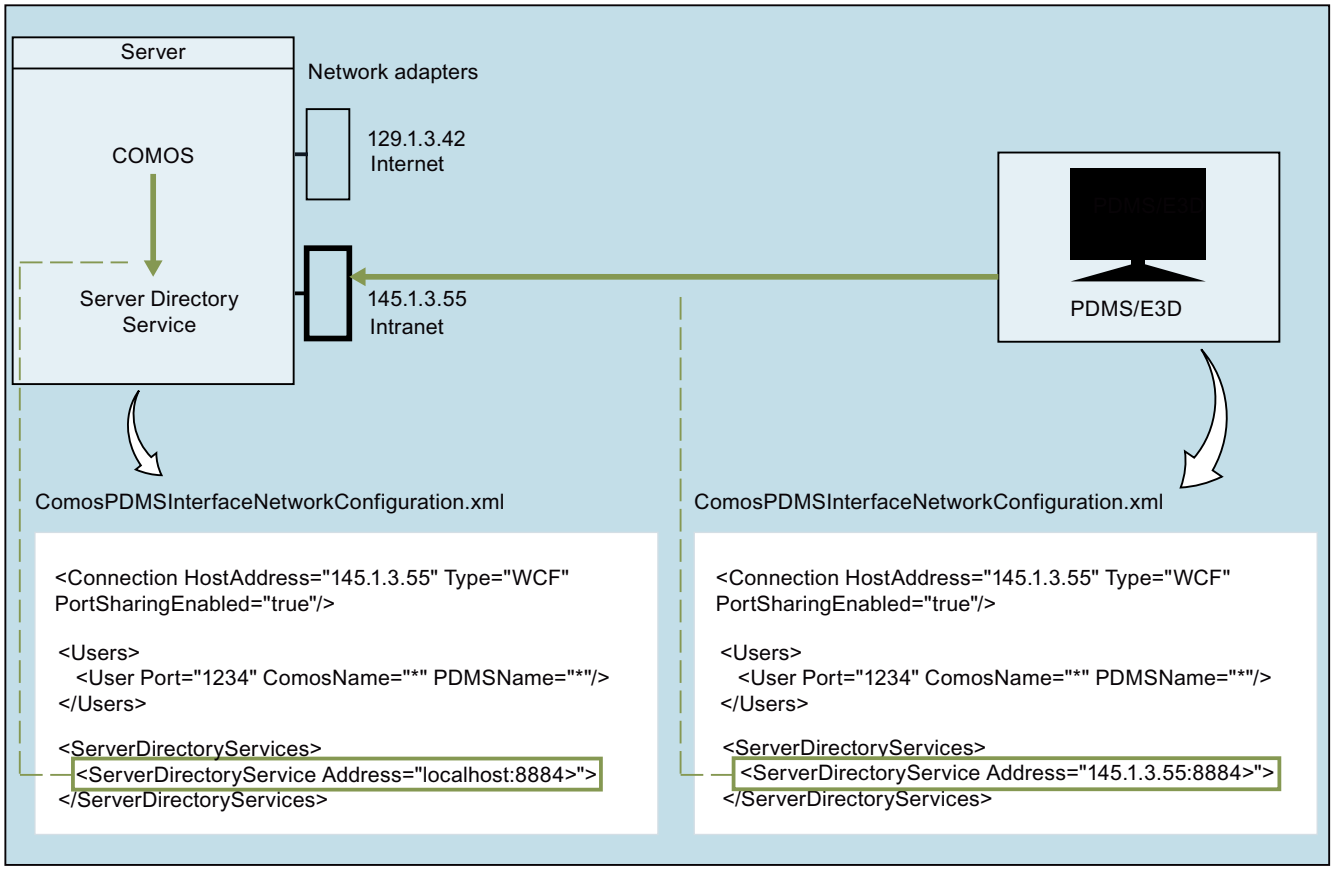
"HostAddress" attribute:

- If you specify the IP address of the network adapter on the server COMOS side in the "HostAddress" attribute, COMOS can only be accessed via this network adapter by PDMS/E3D in the case of a connection without use of the SDS.
- If you specify the IP address of the network adapter on the PDMS/E3D side in the "HostAddress" attribute, PDMS/E3D connects with a COMOS client of the COMOS servers that can be accessed under this network adapter.



"Address" attribute:

- If you specify the value "localhost" on the COMOS side in the "Address" attribute, all COMOS clients of the COMOS server log on at the SDS that is installed on the same computer as the COMOS server.
- If you specify the IP address of the network adapter on the PDMS/E3D side in the "Address" attribute, PDMS/E3D connects with the SDS that can be accessed under this network adapter.



All the COMOS clients that are derived from the COMOS server that logged on to SDS are offered to the PDMS/E3D user to establish a connection.

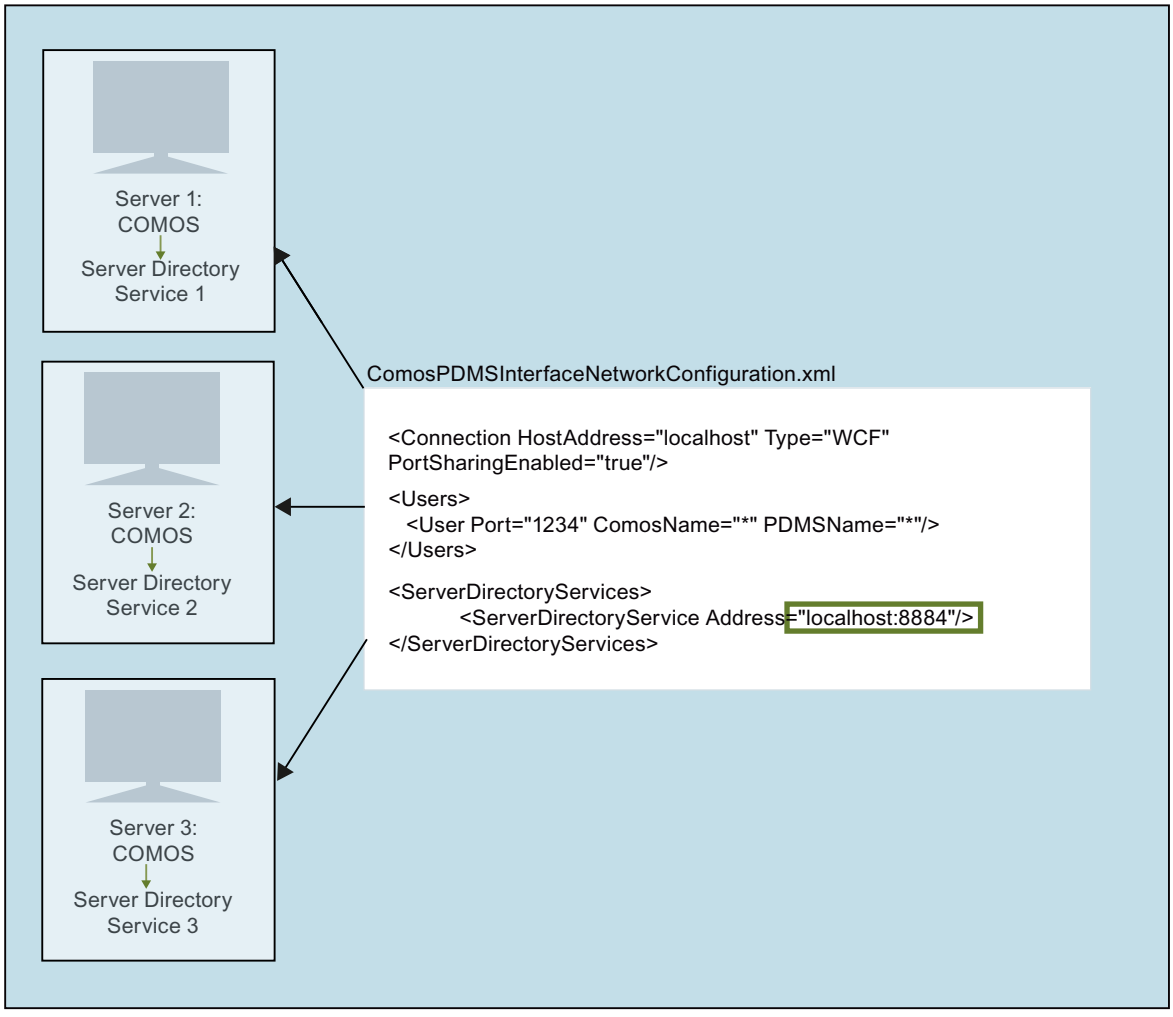
Example 3: Multiple server, multiple SDS (local host)

You can also install the SDS on multiple servers. Depending on the configuration, all the servers can use the same network configuration file.

Network configuration file at the server end (COMOS)

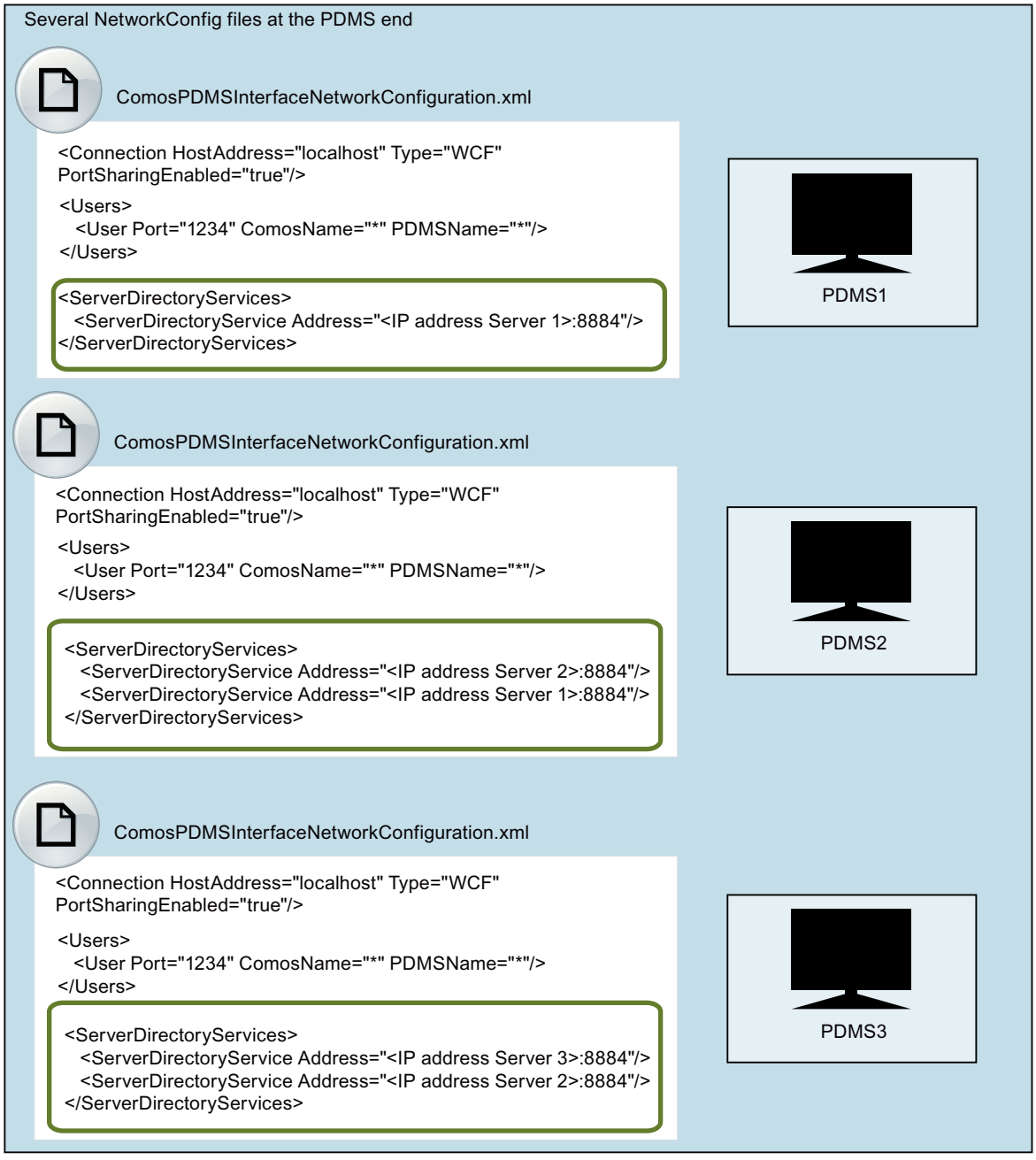
If you specify the following settings, you can use a single network configuration file for all servers:

- In the "Connection" node: HostAddress="localhost"
- In the "ServerDirectoryService" node: Adress="localhost:8884"
  If you specify the value "localhost" in the "Address" attribute, every COMOS client of the COMOS server only registers at the SDS on the same server. If a PDMS/E3D user is connected with one of the SDSs, the user sees only those COMOS clients that have registered at this SDS. COMOS instances that have registered at other SDSs are not offered to the PDMS/E3D user.

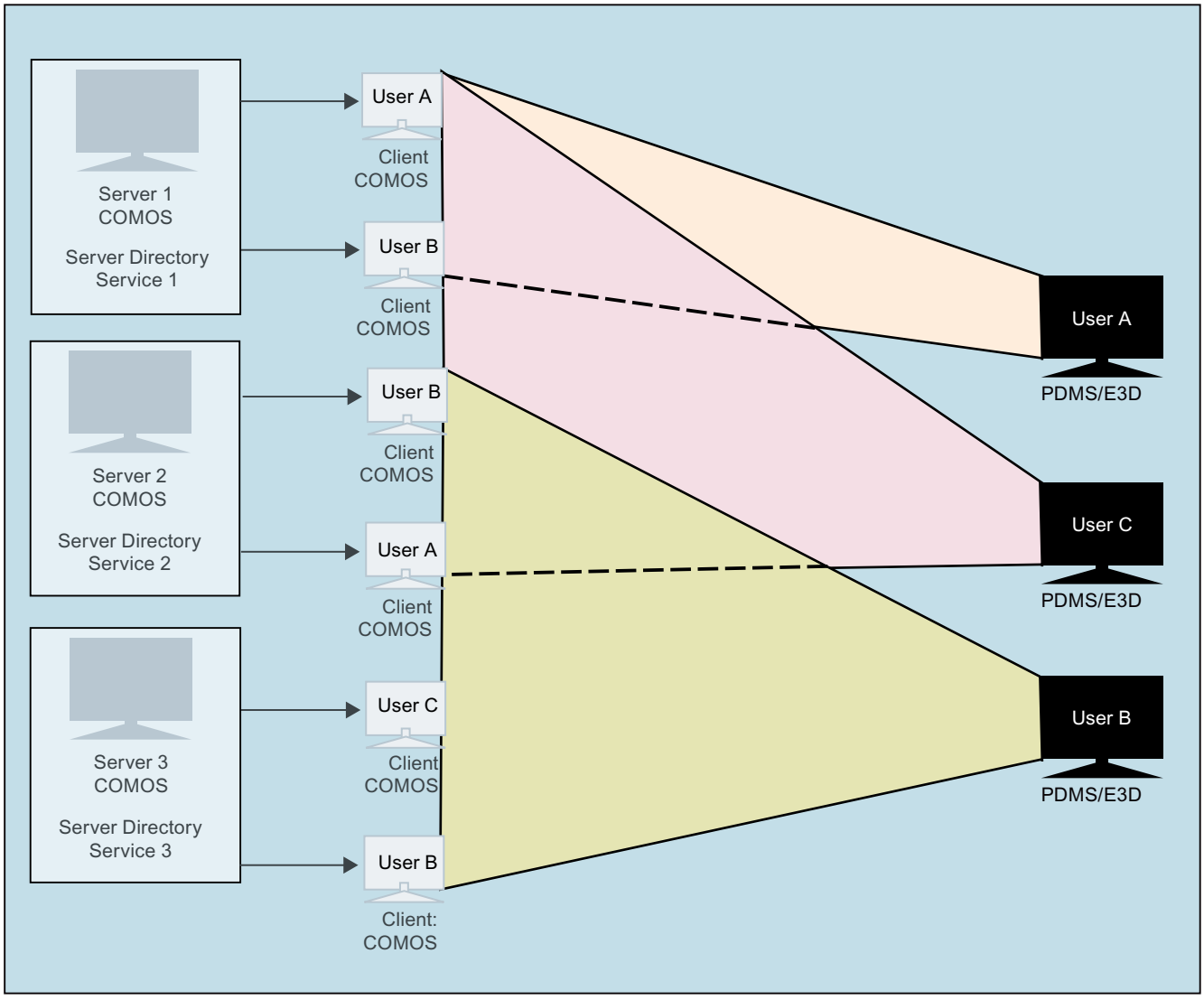

Server 1:
COMOS
Server Directory
Service 1
Server 2:
COMOS
Server Directory
Service 2
Server 3:
COMOS
Server Directory
Service 3
ComosPDMSInterfaceNetworkConfiguration.xml
<Connection HostAddress="localhost" Type="WCF"
PortSharingEnabled="true"/>
<Users>
<User Port="1234" ComosName="*" PDMSName="*"/>
</Users>
<ServerDirectoryServices>
<ServerDirectoryService Address="localhost:8884"/>
</ServerDirectoryServices>

Network configuration files at the client end (PDMS/E3D)

Each PDMS/E3D network configuration file contains the IP addresses of the SDSs at which this PDMS/E3D instance is to ask for free COMOS clients.



Result



Server 1

- User A and User B open a COMOS instance on Server 1.
- Both COMOS instances log on at the local SDS (Server Directory Service 1) with the port 8884.
  Reason: The following is specified for the SDS in the network configuration file of Server 1: Adress="localhost:8884"
- User A opens PDMS/E3D.
- PDMS/E3D asks Server Directory Service 1 which COMOS instances are available.
  Reason: The following is specified in the network configuration file of PDMS/E3D 1: Adress=<IP address Server 1>:8884"
- All the COMOS instances of Server 1 are offered to User A for a connection.

Server 2:

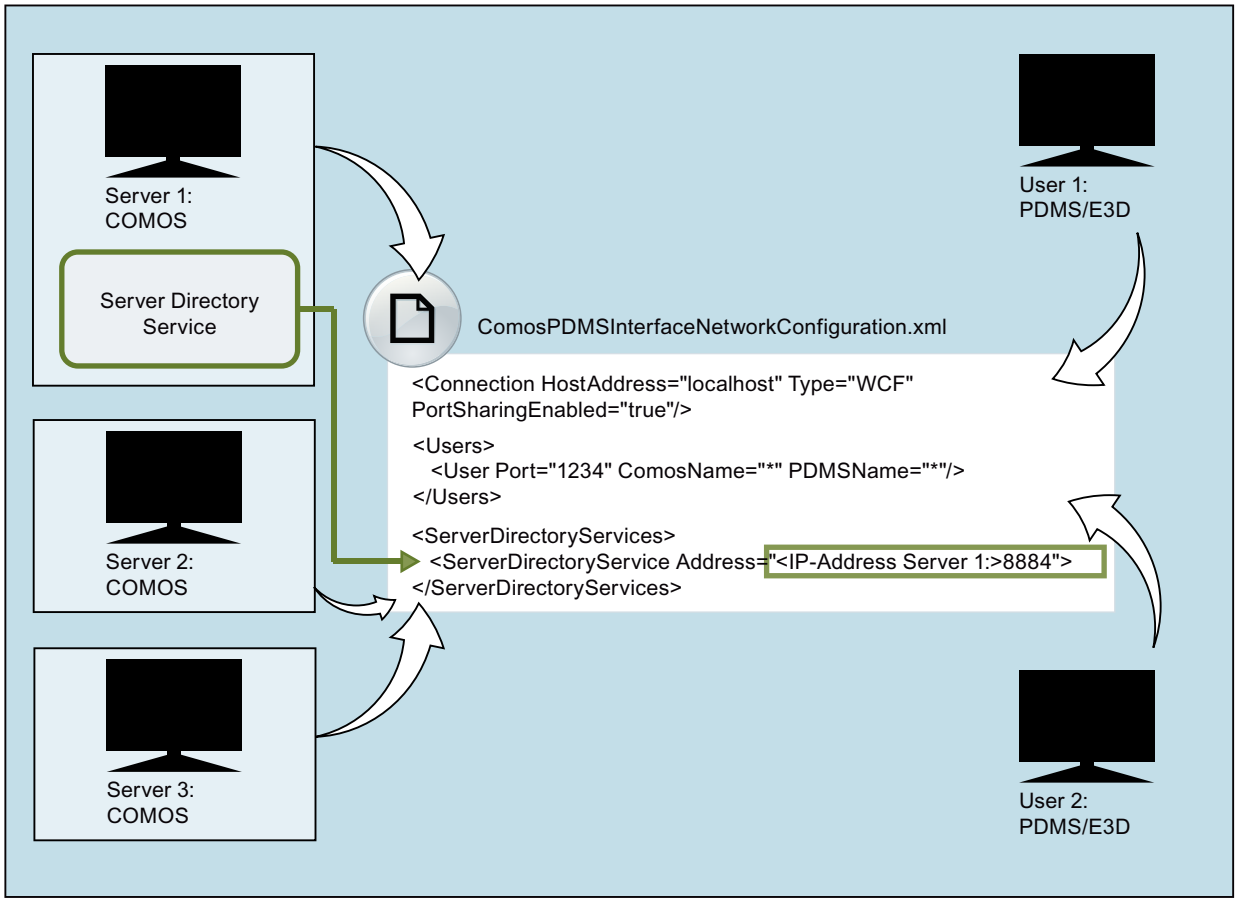
- Similar principle.
- User C is offered all COMOS instances of Server 1 and Server 2 because the SDSs of both servers are specified in the network configuration file of this user's PDMS/E3D installation.

Server 3:

- Similar principle.

Example 4: Several servers, 1 SDS

If a single SDS is to be used by several servers, address the SDS via the IP address of the computer on which it is installed. Since the address is persistent, you can use the network configuration file for all the servers and clients (PDMS/E3D) with the following settings:



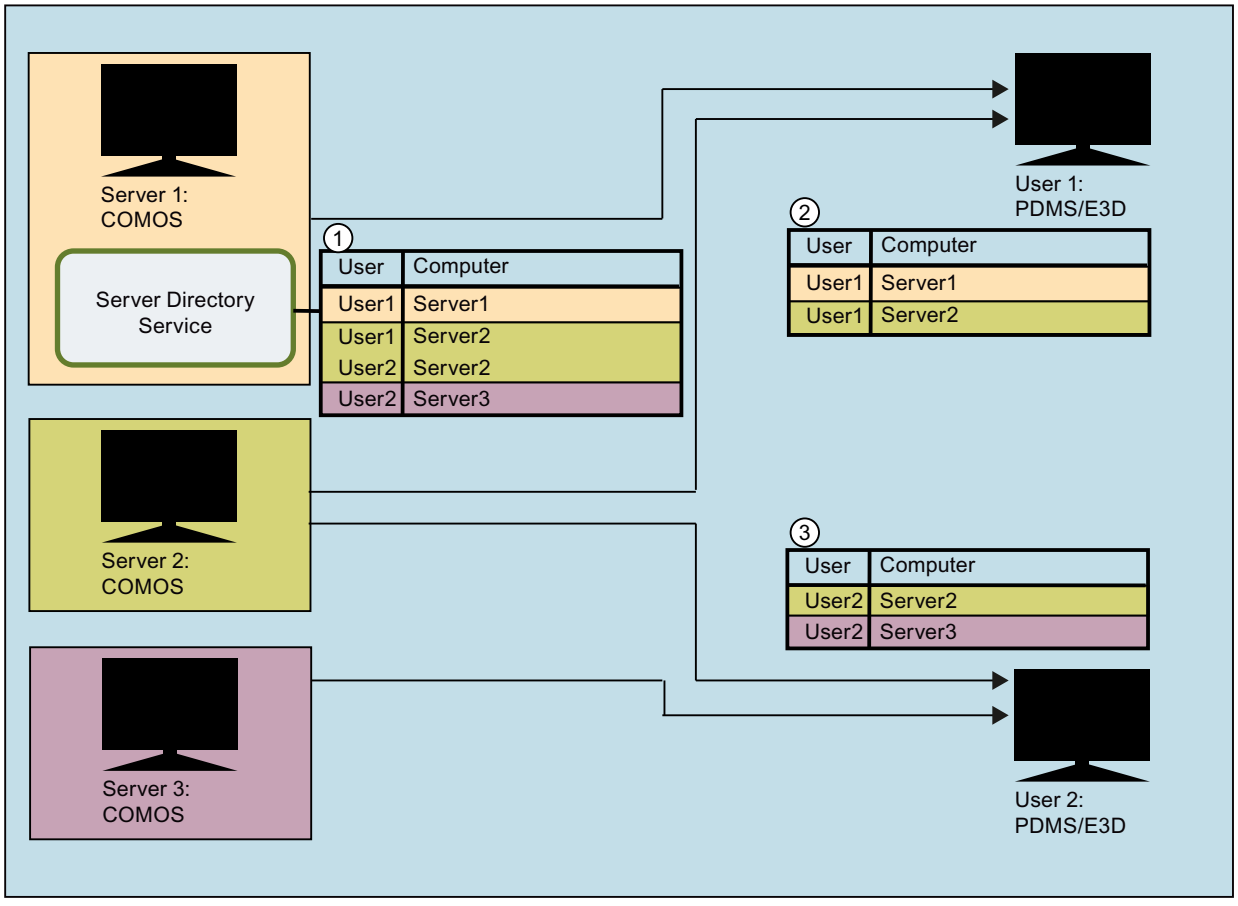
Example:

- One COMOS client is started on Server 1
- Two COMOS clients are started on Server 2
- One COMOS client is started on Server 3

All COMOS clients log on to the SDS. If User 1 or User 2 start PDMS/E3D, all four logged-on COMOS clients are offered for a connection.

User 1 chooses that he wishes to connect to the client of Server 1. The connection is established. User 1 opens a second PDMS/E3D instance. The client already assigned is no longer offered for a connection. User 1 selects a connection to a client of Server 2. The connection is established.

User 2: Analog to User 1



① List of all connections that the SDS has established

② List of all connections on the computer of User 1

③ List of all connections on the computer of User 2

2.3.6.10 Working in multiple networks

Note

If you work in different networks, the Server Directory Service must be hosted on a computer in the network in which the PDMS/E3D instances are located.

Specify one or more hosts of the Server Directory Service in the "ComosPDMSInterfaceNetworkConfig.xml" file.

Requirement

You have started the SDS on a computer in the network in which the PDMS/E3D instances are located.

Procedure

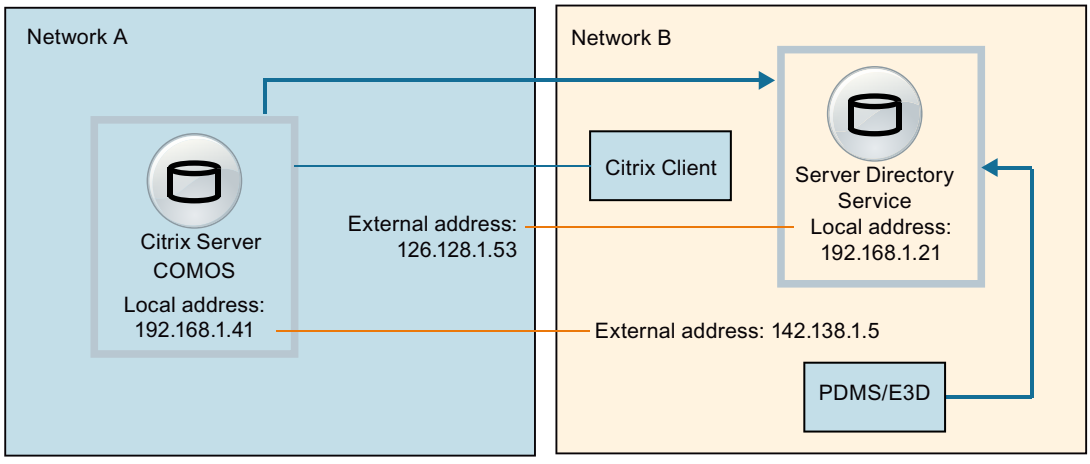
Edit the network configuration files that COMOS accesses. See chapter Preparing the network configuration file for working with the SDS (Page 27).

Edit the network configuration files that PDMS/E3D accesses. See chapter Working in a network (Page 31).

Example

The local address (192.168.1.41) of the Citrix server from Network A is resolved to the external address (142.138.1.5) in Network B.

(This external address is registered in the Server Directory Service. When a PDMS/E3D instance from Network A attempts to connect to a COMOS client, the connection is established via the external address.)



- COMOS network configuration file:
  - HostAddress="192.168.1.41" (local address of the Citrix machine in network A)
  - Address="126.128.1.53 : 8884" (External address of the SDS)
- Network configuration file of PDMS/E3D:
  - HostAddress="142.138.1.5" (external address of the Citrix computer in network A that can be reached from network B)
  - Address="192.168.1.21 : 8884" (Local address of the SDS)

2.3.6.11 Working via Citrix

The way communication works via Citrix and the structure of the network communication files is the same as that for communication in a network. See also chapter Working in a network (Page 31).

Only difference: When working via Citrix, PDMS/E3D users are only offered such COMOS instances for connection that have been started from the same computer as PDMS/E3D.

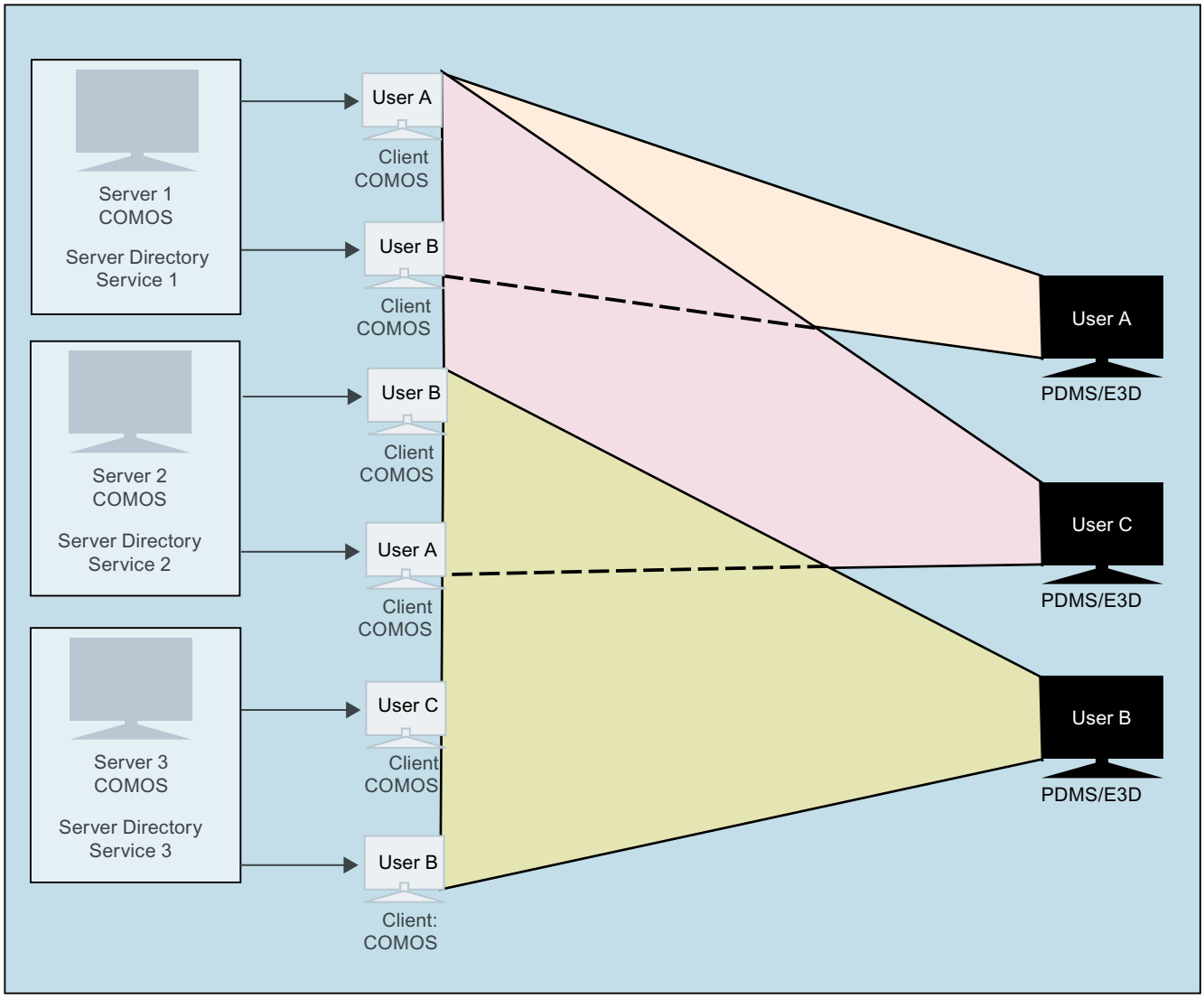
Procedure

Edit the network configuration files which COMOS and PDMS/E3D access. See chapter Preparing the network configuration file for working with the SDS (Page 27).

Example

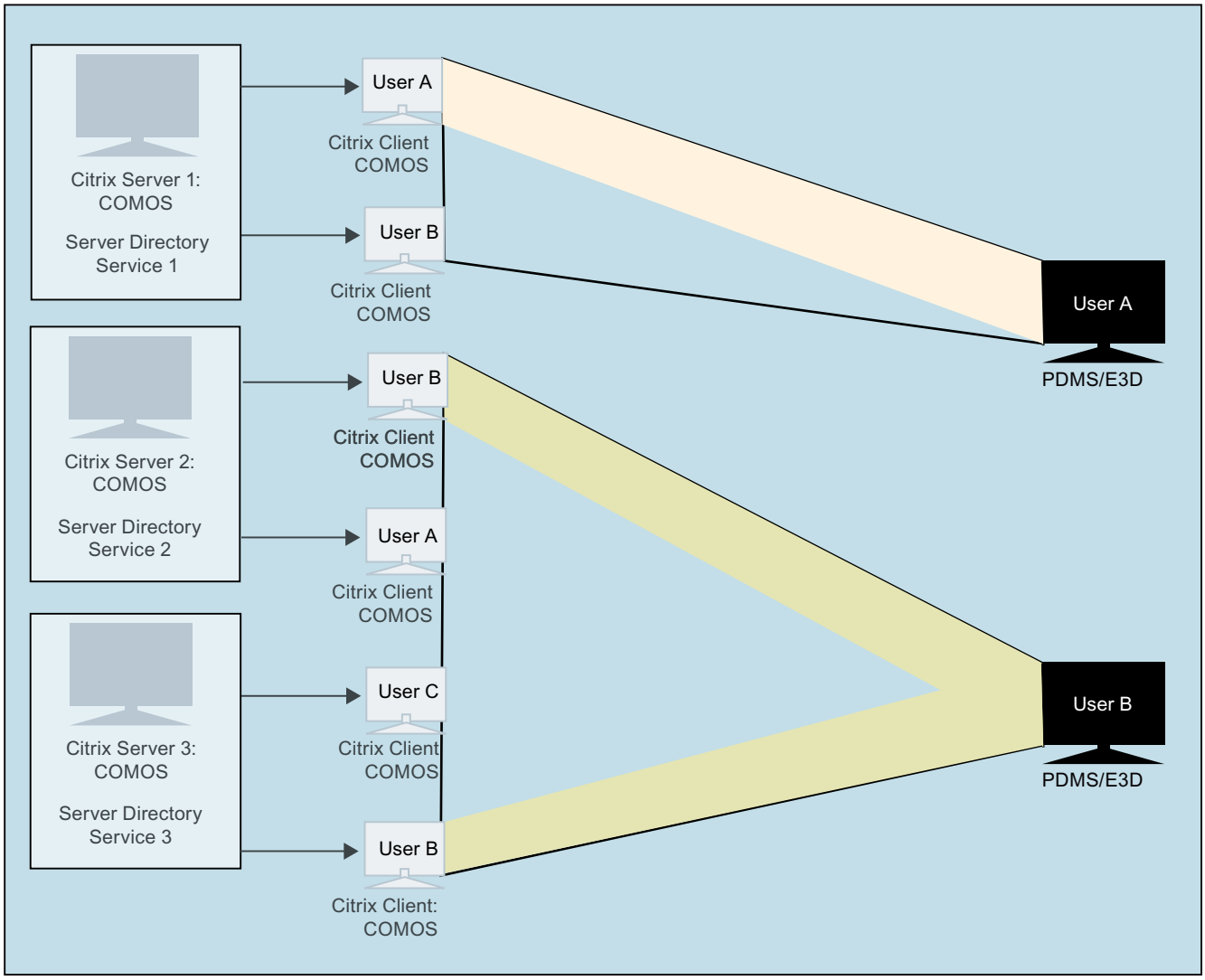
Initial situation: The constellation and content of network communication files correspond to those described for working in a network. See Chapter Example 3: Multiple server, multiple SDS (local host) (Page 34).

- Access options for working in a network:
  Users may connect to all the COMOS clients whose COMOS server logs into the same SDS at which PDMS/E3D also requests.



- Access options for working via Citrix:
  For the purposes of clarity, User C was excluded here.
  - User A may connect to the COMOS clients of Server 1 in accordance with the configuration.
  - User B may connect to the COMOS clients of Server 2 and 3 in accordance with the configuration.

However, since PDMS/E3D may only connect with COMOS clients that were started by the same user in a Citrix environment, only the corresponding clients are offered to User A and B for a connection. The clients offered are highlighted in color.



2.3.6.12 WCF security options

Operation without security options

If you do not make any security settings, data is still encrypted during transmission by standard WCF functionality.

- If you do not make any settings for the authorization, any authenticated PDMS/E3D instance in the same domain can connect to the server on which COMOS is started.
- If PDMS/E3D and the server are located in different domains and a trust status does not exist between the domains, COMOS and PDMS/E3D cannot connect.
- If PDMS/E3D and the server are located in different domains and a trust status exists between the domains, COMOS and PDMS/E3D can connect, if no special settings with regard to the authorization were carried out.

Overview of WCF security options

By default the following security option is preset for the SDS:

- WithoutGroupBasedAuthorization
  User groups are not checked.

You can use the following security options for the SDS and for communication between COMOS and PDMS/E3D:

- ServiceClientUserMustMatchServiceHostUser
  If this option is set, COMOS and PDMS/E3D have to be started under the same Windows user account, so that a connection can be established between PDMS/E3D and COMOS.
- ManualWindowsClientCredentialEntry
  Displays a window in which users must enter their logon information when PDMS/E3D is started. The manually entered access data are used for identification toward the server / SDS.
- GroupBasedAuthorization
  Configures the host of the SDS in such a way that Windows user groups are used for authorization.
- TransportBasedWindowsAuthentification
  The Windows authentication is carried out once when the connection is established.
- MessageBasedWindowsAuthentification
  The Windows authentication is carried out on the message level. It is carried out every time when a message is sent between COMOS and PDMS/E3D.

Combining security options

If the following security options are used together, PDMS/E3D can only connect to the COMOS instance if the COMOS has been started with the same user access as PDMS/E3D.

- ServiceClientUserMustMatchServiceHostUser
- One of the following options:
  - TransportBasedWindowsAuthentification (better performance)
  - MessageBasedWindowsAuthentification (secure data exchange)

If you use the following security settings, the user is prompted at the PDMS/E3Dside to enter a user name and a password when the connection is established between PDMS/E3D and COMOS. The connection is only established if the user enters the same access data as when starting COMOS. The access data can differ from those used when starting PDM/E3D.

- ServiceClientUserMustMatchServiceHostUser
- ManualWindowsClientCredentialEntry
- TransportBasedWindowsAuthentification

2.3.6.13 Specifying WCF security options for the SDS

Note

Security options for SDS COMOS and PDMS/E3D have to agree

If you use security options, the security options for the SDS, COMOS and PDMS/E3D have to agree.

You can specify the security options for the SDS in different ways.

See also Chapter WCF security options (Page 44).

As the start parameter of the SDS

Example:

```
-tcphost localhost -tcpport 8000 -wcfsecurityoptions
"TransportBasedWindowsAuthentification|
ServiceClientUserMustMatchServiceHostUser|GroupBasedAuthorization"
```

In a file in the format "exe.config"

If you have created an "exe.config" file, you can specify the security options directly in this file.

See also Chapter Setting the port and host of the SDS (Page 25).

Note

Protecting configuration files against changes

All the configuration files that contain security settings have to be secured by the Administrator against unauthorized changes.

Example:

```
<configuration>
<appSettings>
 <add key="TcpHost" value="localhost"/>
 <add key="TcpPort" value="8000"/>
 <add key="WcfSecurityOptions"
value="TransportBasedWindowsAuthentification|
ServiceClientUserMustMatchServiceHostUser|
ManualWindowsClientCredentialEntry"/>
</appSettings>
</configuration>
```

In the registry

1. Create a "SOFTWARE\INNOTEC\COMOS\ServerDirectoryService" node under HKEY_LOCAL_MACHINE.
2. Create a "WcfServiceOptions" value of the type string.
3. Specify the security options. Separate the security options with "|".

In a service configuration file

See Chapter Specify the security settings for communication with the SDS via the service configuration file (Page 47).

2.3.6.14 Specifying WCF security options for COMOS and PDM/E3D client

Note

Security options for SDS COMOS and PDMS/E3D have to agree

If you use security options, the security options for the SDS, COMOS and PDMS/E3D have to agree.

Note

Protecting configuration files against changes

All the configuration files that contain security settings have to be secured by the Administrator against unauthorized changes.

You can specify the security options for COMOS and PDMS/E3D in different ways.

See also Chapter WCF security options (Page 44).

In the network configuration files

1. Open the "ComosPDMSInterfaceNetworkConfig.xml" file which PDMS/E3D accesses.
2. In a "WcfSecurityOptions" attribute of the <Connection> node, specify which WCF security options should be used. You can specify a combination of the security options. See Chapter WCF security options (Page 44).
3. Separate the security options with "|".
   Example:
   ```
   <Connection Type="WCF" HostAddress="127.0.0.1"
    WcfSecurityOptions="TransportBasedWindowsAuthentification|
    ServiceClientUserMustMatchServiceHostUser|
   ManualWindowsClientCredentialEntry"
   />
   ```
4. If you use different network configuration files: Open the "ComosPDMSInterfaceNetworkConfig.xml" file which COMOS accesses.
5. Proceed analogously.

In a service configuration file

See also chapter Specify the security settings for communication between COMOS and PDMS/E3D via configuration file (Page 50).

You reference the file in the network configuration file.

2.3.6.15 Specify the security settings for communication with the SDS via the service configuration file

Alternatively to the options described you can specify the security options in a service configuration file. The file allows you to define security settings more finely than with the security options described here by means of WCF default means. See also Chapter WCF security options (Page 44).

- File name: Any
- File format: .config

Content of the file

The file shown here does not contain any security settings. It serves as the basis for your work and can be extended with security-relevant entries.

The file must contain at least the following information:

```
<?xml version="1.0" encoding="utf-8"?>
  <configuration>
    <system.serviceModel>
      <bindings>
        <netTcpBinding>
          <binding
name="MessageSystem.ServerDirectoryService.Binding"/>
        </netTcpBinding>
      </bindings>
      <services>
        <service name="MessageSystem.ServerDirectoryService">
          <endpoint
address="Comos.External.Communication.ServerDirectoryService"
           binding="netTcpBinding"
bindingConfiguration="MessageSystem.ServerDirectoryService.Binding"

contract="Comos.External.Communication.Services.IServerDirectory"/>
        </service>
      </services>
      <client>
          <endpoint
name="MessageSystem.ServerDirectoryService.EndPoint"

address="Comos.External.Communication.ServerDirectoryService"
           binding="netTcpBinding"

bindingConfiguration="MessageSystem.ServerDirectoryService.Binding"

contract="Comos.External.Communication.Services.IServerDirectory"/>
      </client>
    </system.serviceModel>
  </configuration>
```

Editing a service configuration file

The processing of service configuration files is WCF-specific. A documentation of this topic is available on the Microsoft website:

- See Configuring services with configuration files (https://msdn.microsoft.com/en-us/library/ms733932(v=vs.110).aspx).
- See WCF Security Fundamentals (https://msdn.microsoft.com/en-us/library/ff649233.aspx).

The following information must be included:

- <netTcpBinding> node
- Service name: "MessageSystem.ServerDirectoryService"
- Client endpoint configuration name: "MessageSystem.ServerDirectoryService.Endpoint"
- Relative endpoint address: "Comos.External.Communication.ServerDirectoryService"
- Contract name: "Comos.External.Communication.Services.IServerDirectory"

Integrating the service configuration file

- Option 1: If you have created an "exe.config" file, you can integrate the service configuration file there.
  Example:
  ```
  <configuration>
  <appSettings>
   <add key="TcpHost" value="localhost"/>
   <add key="TcpPort" value="8000"/>
   <add key="WcfServiceConfigurationFile"
   value="ServerDirectoryService.config"/>
  </appSettings>
  </configuration>
  ```
  See also Chapter Setting the port and host of the SDS (Page 25).
- Option 2: Including as a start parameter
  Example:
  ```
  -tcphost localhost -tcpport 8000 -wcfserviceconfigurationfile
  ServerDirectoryService.config
  ```
  See also Chapter Setting the port and host of the SDS (Page 25).
- Option 3: Including in the registry
  - Under "HKEY_LOCAL_MACHINE", create a "SOFTWARE\INNOTEC\COMOS\ServerDirectoryService" node.
  - Create a "WcfServiceConfigurationFile" value of the string type.
  - Enter the path to the service configuration file.

2.3.6.16 Specify the security settings for communication between COMOS and PDMS/E3D via configuration file

Alternatively to the options described you can specify the security options in a service configuration file. The file allows you to define security settings more finely than with the security options described here by means of WCF default means. See also Chapter WCF security options (Page 44).

- File name: any
- File format: .config

Content of the file

The file shown here does not contain any security settings. It serves as the basis for your work and can be extended with security-relevant entries.

The file must contain at least the following information:

```
<configuration>
 <system.serviceModel>
    <bindings>


      <netTcpBinding>
        <binding name="MessageSystem.MessengerService.Binding"
maxReceivedMessageSize="2147483647">
          <readerQuotas maxStringContentLength="2147483647"/>
        </binding>
      </netTcpBinding>


    </bindings>
    <services>
       <service name="MessageSystem.MessengerService">
          <endpoint
address="Comos.External.Communication.MessageSystemWcf"
        binding="netTcpBinding"

bindingConfiguration="MessageSystem.MessengerService.Binding"

contract="Comos.External.Communication.Services.IMessenger"/>
       </service>
     </services>
     <client>
          <endpoint name="MessageSystem.MessengerService.EndPoint"
```

```
                    address=
"Comos.External.Communication.MessageSystemWcf"
binding="netTcpBinding"
bindingConfiguration="MessageSystem.MessengerService.Binding"

contract="Comos.External.Communication.Services.IMessenger"/>
     </client>
 </system.serviceModel>
</configuration>
```

Editing a service configuration file

The processing of service configuration files is WCF-specific. A documentation of this topic is available on the Microsoft website:

- See Configuring services with configuration files (https://msdn.microsoft.com/en-us/library/ms733932(v=vs.110).aspx).
- See WCF Security Fundamentals (https://msdn.microsoft.com/en-us/library/ff649233.aspx).

The following information must be included:

- <netTcpBinding> node
  Maximum message and string size: "2147483647"
  This is the maximum value that can be represented as a signed 32-bit integer
- Service name: "MessageSystem.MessengerService"
- Client endpoint configuration name: "MessageSystem.MessengerService.Endpoint"
- Relative endpoint address: "Comos.External.Communication.MessageSystemWcf"
- Contract name: "Comos.External.Communication.Services.IMessenger"

Integrating the configuration file

1. Open the "ComosPDMSInterfaceNetworkConfig.xml" file.
2. In a "WcfServiceConfigurationFile" attribute in the <Connection> node, enter the path to the file.
   Example:
   ```
   <Connection Type="WCF" HostAddress="127.0.0.1"
   WcfServiceConfigurationFile="MessengerService.config"/>
   ```

2.3.6.17 Extending the configuration files for SDS

When you define the security settings for communication via configuration files, you have to extend the configuration files.

Note

The configuration files cannot be used without the extensions.

Affected chapter:

Specify the security settings for communication with the SDS via the service configuration file (Page 47)

Specify the security settings for communication between COMOS and PDMS/E3D via configuration file (Page 50)

Extensions:

- "receieveTimeout" attribute in the "binding" element
  Example: `<receiveTimeout="1.00:00:00">`
  You can change the value.
  If a PDMS/E3D instance does not send a query to COMOS within the period specified here, the connection is disconnected automatically.
  Example: If a user starts PDMS/E3D and connects to COMOS, the connection is disconnected automatically if the user does not carry out any action in the interface within the specified period.
  If you do not define the security options via configuration files, the timeout amounts to 1 day.
- "reliableSesssion" element
  `<reliableSession enabled="true"/>`
  The element "reliableSession" activates a reliable connection session.

  Note

  Do not change the "true" value

  The value of the "reliableSession" element must always be "true".

  If you do not define the security options via configuration files, a reliable connection session is activated by default.

Example of the netTcpBinding element in the configuration file for COMOS and PDMS/E3D with the specified extensions:

```
<netTcpBinding>
<binding name="MessageSystem.MessengerService.Binding"
maxReceivedMessageSize="2147483647"
receiveTimeout="1.00:00:00">
<readerQuotas maxStringContentLength="2147483647"/>
```

```
</binding>
<reliableSession enabled="true"/>
</netTcpBinding>
```

Example of the netTcpBinding element in the configuration file for the SDS with the specified extensions:

```
<netTcpBinding>
 <binding name="MessageSystem.ServerDirectoryService.Binding"
receiveTimeout="1.00:00:00"/>
 <reliableSession enabled="true"/>
 </netTcpBinding>
```

2.3.6.18 Using user groups

Use user groups to ensure that only certain users have access to the SDS.

Procedure

1. Create a Windows user group for users who are to use the SDS.
   The name of the user group must be "Comos.ServerDirectory.User".
2. Determine the users of the user group.
3. Use the "GroupBasedAuthorization" option when setting the WCF security options. See also chapter Specifying WCF security options for the SDS (Page 45).

2.3.6.19 Using PDMS Engineering Interface with SDS

Requirement

You have performed all settings described.

Procedure

1. Open the PDMS Engineering Interface in COMOS.
2. Open the PDMS Engineering Interface in PDMS/E3D.
   The "Select server" window opens. It displays all unassigned COMOS instances to which you have access.
3. Select a COMOS instance to which PDMS/E3D should connect.

Result

COMOS and PDMS are interconnected.

The selected PDMS/E3D instance is no longer available to other users as long as you are using it.

2.4 Configuring COMOS PDMS/E3D Engineering Interface

2.4.1 Entering COMOS PDMS/E3D Engineering Interface as an add-in

Procedure

1. In the PDMS/E3D installation directory, open the "DesignAddins.xml" file.
2. Enter the following node in the XML file:
   `<string>Comos.PDMSInterface.ComosAddin</string>`

Result

You can open the COMOS PDMS/E3D Engineering Interface. You can find additional information on this topic in the "3D Integration Operation" manual, keyword "Opening the COMOS PDMS/E3D Engineering Interface".

2.4.2 Restricting COMOS objects for the COMOS PDMS/E3D Engineering Interface

With the help of queries, you specify for each user or a group of users which objects a user can transfer from COMOS to the COMOS PDMS/E3D Engineering Interface.

2.4.2.1 Creating queries

Procedure

1. Open the "Attributes > General" tab in the properties of the "Folders for class/subclass definitions" engineering object.
2. In the "Folder for scope queries" field, enter the directory in which you want to create the queries.
3. Under the specified directory, create a query for each user or user group that defines the COMOS objects that will be loaded to the COMOS PDMS/E3D Engineering Interface for the user.
   The query is called when the user clicks on "Request data" in the interface and then enters the name of the query.
4. Assign a query to each user. See also chapter Assigning queries to users (Page 55).

Result

The following COMOS objects are loaded to the interface when the query is launched:

- Pipes
- Equipment
- Components that are defined in the query used and whose owners are located on the P&ID.

2.4.2.2 Assigning queries to users

Requirement

- You have created queries that assemble an object set to be made available in the 2D working area of the PDMS/E3D Engineering Interface. See also chapter Creating queries (Page 54).
- The queries are located in the directory that you specified in the "Folder for scope queries" field on the "Attributes > General" tab at the "Folder for class/subclass definitions (PDMS)" object.

Procedure

Using a PML function named `!!SAGGetScopeID()`, define which query should be called for each user or user group for collecting objects.

Result

If a user clicks the "Request data" button in the COMOS PDMS/E3D Engineering Interface, the query assigned to that user is called.

The determined COMOS objects are shown in the interface in the "COMOS objects" area.

2.4.3 Using classes and subclasses

When working in the COMOS PDMS/E3D Engineering Interface, you can also use some functionalities of the classes and subclasses. The configuration and application is the same as that of the COMOS PDMS Interface.

Functionalities available for the COMOS PDMS Interface:

- Name directories
  See also chapter Name directory (Page 93).
  - Local variables
    See also chapter Working with String Parameters in the name directory (Page 93).
  - Global variables
    See also chapter Defining global variables (Page 79).
- Name mapping
  See also chapter Configuring site mapping and location mapping in name mapping (Page 100).
- Restriction rules for owners
  See also chapter Specifying restriction rules for owners (Page 89).
- Models
  See also chapter Configuring the model (Page 91).

2.4.4 Implement PML macro !!SAGPickUpQueryResult(!p is STRING)

The macro "!!SAGPickUpQueryResult(!p is STRING)" is called automatically once you have executed the macro "!!SAGRunCOMOSQuery()" and generated a CSV file. You can find additional information on this topic in the "3D Integration Operation" manual, keyword "Executing COMOS query in PDMS/E3D".

The absolute path to the CSV file is transferred as argument to the macro "!!SAGPickUpQueryResult(!p is STRING)" in this process.

Procedure

1. Create a macro with the name "!!SAGPickUpQueryResult(!p is STRING)".
   The argument !p includes the absolute path to the CSV file as string.
2. Define how the query results are to be processed further or displayed.

2.4.5 Integrating custom icons for objects in the 3D workspace

You can define your own icons if you do not want to use the standard icons in the 3D working area.

Procedure

1. Open the exchange directory.
   You can see the exchange directory path in the project properties on the tab "PDMS/E3D Engineering Interface".
2. Navigate to the folder "Icons". It is in the file directory in the folder "config" along with the file "Subclasses.xml". Create it if it does not exist.
3. Add graphics to the folder "Icons". The files must have the same name as the PDMS type of the object for which the icon is to be displayed.

2.4.6 Saving attributes of a PDMS/E3D model

As user or administrator, you have the option of saving the attributes of a PDMS/E3D model in a file.

Requirement

- A "ReviewPath" is defined in the "COMOSPDMSInterfaceConfiguration" file.
- You have opened PDMS/E3D, COMOS is available as add-in.
- A PDMS/E3D model of a 2D planning from COMOS is open.

Procedure

1. From the COMOS menu, select the command "Attribute dump".

Result

- The attributes are saved at the default location under "ReviewPath" as TXT file.
- The exported data is available in a format that can be read by COMOS Walkinside, for example.

2.5 Configuration basics for interface operations

2.5.1 Interface objects

Definition of Interface Objects

Interface objects are engineering objects which can be processed by the interface operations.

An object counts as an interface object if it has the following properties:

- In COMOS:
  - "External 3D interface" tab
  - Assignment to a class
  - Assignment to a subclass
    The class must differ from UNDEF.
  - PDMS element type
- In PDMS: UDA ":ComosSClass"

If the devices of your device catalog have these properties, they are processed by COMOS PDMS Interface.

See also

Configuring COMOS interface objects (Page 123)

2.5.2 Classes and subclasses

Definition

There are two groups of classes and subclasses:

1. Classes and subclasses which categorize interface objects
2. Classes and subclasses which are used to configure the execution of an interface operation

Group 1

The following classes and their subclasses belong to this group:

- "Cables (PDMS)":
  Interface objects with this class are used for example as:
  - Pipe or pipe branch
  - HVAC
  - DUCT
  - CABLES
- "Devices with label (PDMS)":
  Interface objects with this class are equivalent to PDMS objects which can be positioned in 3D space.

You can use subclasses to differentiate the behavior of interface objects on the same class. The subclass determines how an interface object is processed during an interface operation.

Group 2

The following classes and their subclasses belong to this group:

- "Queries (PDMS)" class and their subclasses: They are used to configure COMOS queries and make them accessible and executable in PDMS.
- "Document (PDMS)" class and its subclasses: A DocLink is generated for the interface object.
- "MTO (PDMS)" class: Configure the import of MTO data from PDMS/E3D to COMOS. This class has no subclasses.
  See also chapter Configure 'MTO (PDMS)' class (Page 92).

Properties of classes

Classes have the following properties:

- Classes structure the subclasses.
- The number and name of classes are fixed. They are specified by the following standard table:
  "Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A01 Classes"
- A class can have any number of subclasses.

Properties of the subclasses

Subclasses have the following properties:

- The function performed by a subclass is determined by the class to which it belongs. See above.
- The number and names of subclasses are specified by the subclass definition objects and are dynamically saved in the following standard table:
  "Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A02 Subclasses"

2.5.2.1 Class definition objects

Function

Class definition objects are engineering objects which are located directly below the "Folders for class/subclass definitions" object.

Class definition objects have the following functions:

- They represent a class from the following standard table:
  "Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A01 Classes"
- They structure the subclass definition objects.

See also

Creating a class definition object (Page 84)

Standard tables for classes and subclasses (Page 77)

2.5.2.2 Subclass definition objects of "Cables" and "Devices with labels" classes

Function

The subclass definition objects of classes "Tagged item" and "Cable" fulfill the following functions:

- They represent the subclasses of these classes in the engineering view.
- They are used to save and configure the properties of the corresponding members of this subclass.

Defining interface object properties

One subclass definition object defines the following properties for all interface objects of the subclass:

- PDMS element types
- Base object that is used when the interface object is created in COMOS
  The user defines two base objects:
  - One is used when the interface object is created in the unit tree
  - One is used when the interface object is created in the location tree
- Structural behavior:
  - Whether the PDMS interface object is connected to a COMOS object from the unit tree or from the location tree
  - Whether the COMOS interface object is to have a pointer to another COMOS object and whether this object is located in the unit tree or in the location tree
- Regarding name mapping:
  - Generation of the object name on PDMS
  - Rule governing how the interface for a given PDMS object finds the location at which the corresponding COMOS object will be created in the COMOS structure
- Regarding the name directory:
  - Values that are evaluated by the interface operations when you process the interface object
  - Assignment between COMOS attribute and PDMS attribute
  - String parameters that are available in the subclass.
- Regarding the owner restriction rules: Conditions that the owner must fulfill so that an interface object can be created below it
- Which 3D model the interface object obtains
- How the object is initialized during the export if "Function" is set as model

See also

Creating subclass definition objects (Page 84)

Configure subclass definition objects of classes "Cable" and "Tagged item" (Page 85)

2.5.2.3 Subclass definition objects of the "Queries" class

Working with COMOS queries in PDMS

The interface allows users to work with COMOS queries in PDMS. Users have the following options:

- They select a query in PDMS from a range of predefined COMOS queries.
- Optional: They define filters for the query.
- Optional: They define the start object.
- They start the query in PDMS.
- They see the result of the query in PDMS or in an external program such as Excel.

This functionality is implemented with the help of COMOS queries and subclass definition objects of the "Query" class.

The subclass definition objects of the "Query" class define the following:

- Queries that are available in PDMS
- Base objects of the objects in the result list
- Preset start object
- Output format in which the result list will be transferred

See also

Classes and subclasses (Page 58)

Creating subclass definition objects (Page 84)

Creating and configuring query subclasses (Page 115)

2.5.2.4 Subclass definition objects of the "Documents" class

The interface allows users to create DocObjs for draft objects in COMOS.

Requirement

- The draft objects have a DDNM attribute.
- The DDNM attribute references a design object which is connected to a COMOS object.
- A COMOS document under which the DocObjs are imported has been specified in the draft structure.

Function

In the subclass definition objects of the "Document" class, you define all settings which are necessary to create DocObjs for draft objects.

These include:

- Place in the PDMS draft structure at which the COMOS document for which DocObjs will be created is specified.
  As you are able create any number of subclass definition objects, you can configure the interface in such a way that "Import DocLinks" can be started at different levels of the draft structure according to the requirements.
- Filter criteria that restrict the COMOS document for which the DocObjs can be created.
- Setting indicating whether DocObjs are created only for draft objects with names or also for draft objects without names.
- Owner restrictions for the draft objects for which DocObjs are created.

See also

Creating subclass definition objects (Page 84)

Configuring "Import DocLinks" (Page 118)

Standard tables for classes and subclasses (Page 77)

2.5.2.5 "Folders for class/subclass definitions" object

Path in the data base: "@20 > D30 > A20 > Y00R00033 @PDMSMAP Folder for class/subclass definitions (PDMS)"

Function

This object fulfills the following functions:

- At this object, you make general settings which apply to the entire interface and not only to a specific class or subclass.
- The class and subclass definition objects are managed below the object.
- In the context menu of the object, you can save the configuration of the subclass definition objects in a file and refresh the standard table used to save the subclasses.

Overview of the general interface settings

The following general interface settings are configured in the "Folders for class/subclass definitions" object:

- Interface operations after which feedback messages are displayed to users.
- Use and configuration of the site mapping and zone mapping.
  See also chapter ZONE mapping and SITE mapping (Page 70).
- Configuration of the character mapping
  See also chapter Character mapping (Page 69).

- Configuration of the unit mapping
  See also chapter Units mapping (Page 69).
- Registered pre/post executables
  See also chapter Pre/post executables (Page 69).
- Global String parameters
  See also chapter String Parameters (Page 66).

See also

Creating the "Folders for class/subclass definitions" object (Page 77)

Synchronizing classes and subclasses in COMOS and PDMS (Page 120)

Operation messages (Page 70)

2.5.3 Name mapping

Function

Name mapping fulfills the following functions:

- It generates a PDMS/E3D name for a COMOS interface object.
  The interface uses the PDMS/E3D name to identify the PDMS/E3D interface object to which the COMOS object is connected.
- It uses the PDMS/E3D name of a PDMS/E3D interface object to ascertain the path to the COMOS interface object to which it is connected.
- If the structural behavior of the subclass definition object prescribes that a COMOS interface object references another COMOS object, the name mapping derives the referenced object from the PDMS/E3D name.
- Optional:
  If the COMOS interface object that is to be connected to a PDMS/E3D object does not exist yet: The name mapping determines the base object used to create the COMOS interface object.
- If the owner structure of the COMOS interface object that is to be connected to a PDMS/E3D object is incomplete or missing, the name mapping specifies which owners are created.

Site mapping and location mapping

The unit tree and the location tree do not usually share the same object structure. This is why it is necessary to manage two mapping tables:

- Site mapping: Table which defines name mapping in the unit tree
- Location mapping: Table which defines name mapping in the location tree

You can find information on the structure of the tables, their configuration, and the algorithm on which the name mapping is based in chapter Configuring site mapping and location mapping in name mapping (Page 100).

See also

Structural behavior (Page 65)

Level rules of name mapping (Page 64)

2.5.3.1 Level rules of name mapping

Function

You can define deviations from the standard evaluation model for site mapping and location mapping.

These deviations are particularly required in the following cases:

- Your project uses an alias structure.
- Your project uses different hierarchy depths.

Rule "Take alias rather than object"

If this rule is activated, the alias structure is used to generate the PDMS names.

If no alias is available, the name or description of the object is used.

Rule "Skip level if data part empty"

If this rule is activated, you can map hierarchies of different depths in name mapping. The maximum number of mapping entries in the name mapping is determined by the hierarchy with the deepest level.

If the evaluation of the data item or fixed name of a mapping entry returns an empty string, the process continues with the next mapping entry.

Application example: Your equipment is located in the "Equipment" folder. Only pumps are not directly located in the folder; these are to be found in its "Pumps" subfolder.

Rule "Skip level if unable to create object"

If a COMOS object cannot be created because of its configuration - because the potential owner only permits certain elements as subobjects, for example - the name mapping does not abort. It jumps to the next mapping entry. The entry is processed with the next PDMS name part.

Result: This rule allows you to use name mapping in COMOS to create object structures with hierarchy levels of different depths.

Rule "Use label for fixed name"

If this rule is activated, the label of a COMOS object rather than its name is used in the "Fixed name" range.

See also

Configuring level rules (Page 109)

Name mapping (Page 63)

2.5.3.2 Structural behavior

Function

The structural behavior of a subclass definition object defines the following points:

- Creation mode: Whether the COMOS interface object to be connected to the PDMS object is located in the unit tree or in the location tree
- Assign mode: Whether the COMOS interface object references another COMOS object and whether the referenced object is located in the unit tree or in the location tree

Depending on how you define the structural behavior, either site mapping or location mapping is evaluated during name mapping to fulfill one of the following tasks:

- Find or create an interface object in order to connect it with a PDMS object.
- Find or create the COMOS object referenced by the COMOS interface object.

See also

Name mapping (Page 63)

Working with area pipes (Page 65)

Defining structural behavior (Page 86)

2.5.4 Working with area pipes

Definition

An area pipe is an object which is generated whenever an interface operation processes an object whose subclass has the following structural behavior:

- Creation mode: "Create in location tree"
- Assign mode: "Assign to object in unit tree"

The aim of this setting is to map multiple PDMS objects to a single COMOS object. For this purpose, the area pipe level is inserted between the PDMS objects and the COMOS object.

Application case

A pipe runs across different zones in PDMS, meaning that there are several pipe objects, but logically these represent the same pipe. There is only one pipe object in the COMOS unit tree.

Area pipes are created for the PDMS pipes in the location tree and connected to the PDMS pipes. The area pipes get a reference to the same COMOS pipe object.

Properties

Area pipes have the following properties:

- Area pipes are connected to the PDMS object.
- Area pipes get a unit link to an interface object from the unit tree.
- The full name of the corresponding PDMS object is written to the "Name" property of the area pipe.
  In contrast, only the last name part of the PDMS name is written to the name of the referenced unit object. Depending on how the name mapping is configured, the name part can even be truncated before it is written to the "Name" property.
- Data exchange can take place between the unit object and the assigned PDMS objects.

See also

Structural behavior (Page 65)

2.5.5 String Parameters

Definition

A String parameter is a variable which returns or processes a string:

- A String parameter reads a COMOS value or a fixed string. If the String parameter is configured accordingly, it writes the value to PDMS.
- A String parameter reads out a PDMS value or a fixed string. If the String parameter is configured accordingly, it writes the value to PDMS.

Global String parameters

Availability:

These String parameters are available everywhere in the interface operations area. They are defined at the "Folders for class/subclass definitions" object.

In PDMS, global String parameters are evaluated independently of the current element.

In COMOS, they are evaluated at the following objects:

- In the project
- In the workset
- At the "Folders for class/subclass definitions" object

Purpose:

- To define variables which are globally available in the interface operations area.
- To prevent interface operations from being executed if a COMOS value and a PDMS value defined by the String parameter do not match.

Local String parameters

Availability:

These String parameters are only available for subclasses. They are defined in the subclass definition objects and evaluated at the interface object.

Purpose:

- To define variables that are locally available in a subclass.
- To perform one of the following actions during the interface operations:
  - Export: Write a COMOS value to PDMS.
  - Import: Write a PDMS value to COMOS.
  - Check: Compare a COMOS value with a PDMS value.

  Without these actions it is not possible to make an assignment between the attributes of an interface object in COMOS and PDMS.
- Optional: To stop objects from being created for the export if a COMOS value and a PDMS value are different.

See also

Name directory (Page 93)

Working with String Parameters in the name directory (Page 93)

Defining global variables (Page 79)

2.5.6 Restriction rules for owners

Definition

With the restriction rules for owners you can define rules for the subclass definition objects. The rules determine which owners are permitted in PDMS for objects of this subclass.

The interface evaluates the restriction rules for owners before it creates an object in PDMS or makes an assignment between a COMOS object and a PDMS object. If an action violates one of the rules, the object is not created or no assignment is made.

Example

With the restriction rules for owners, the following rules can be implemented, for example:

- In PDMS, the exported interface objects are only created under zones whose purpose attribute is set to "TAG".
- In PDMS, pipes and branches are only created under zones whose purpose attribute is set to "PIP".
- In PDMS, the equipment is only created under zones whose purpose attribute is set to "EQU".

2.5.7 Model information

Model types

The following model types are available for interface objects:

- Design template:
  - The model is defined via a design template. The template has been imported from PDMS to COMOS.
  - The user defines one main template and up to five templates for secondary equipment.
- PDMS element:
  The model of an existing PDMS element is used.
- Function:
  The model is defined via a PML function.
- None:
  A box is created.

Possible sources

You have the following options for defining which model type an interface object uses:

- The model is inherited from the base object
- The model is taken from the subclass definition object
- The settings inherited from the base object or the subclass definition object are over-defined in the properties of the interface object.

Export

When an interface object is exported to PDMS, the model information defined in COMOS is also exported.

The object is displayed in PDMS with the corresponding model.

See also

Configuring the model (Page 91)

Overwriting the inherited model information (Page 124)

2.5.8 Pre/post executables

Definition

Pre/post executables are user-defined PML functions which are called prior to starting or after completion of an interface operation.

- Pre executables execute steps serving the purpose of initialization and have an effect on the entire interface operation.
- Post executables execute concluding steps.

See also

Using pre/post executables (Page 79)

2.5.9 Character mapping

Maintaining the naming convention

Some characters, such as blanks, are not allowed to form part of a PDMS name.

If the name mapping in COMOS generates a PDMS name, the generated string may contain one or more characters that are not permitted in PDMS.

You define which characters are to replace these illegal characters in PDMS in the character mapping of the "Folders for class/subclass definitions" object.

See also

Using character mapping (Page 81)

2.5.10 Units mapping

Unit conversion

It may be that an attribute was assigned a different unit in COMOS than in PDMS.

Units mapping enables you to map these units. When an attribute of this type is exported or imported, the attribute value is converted as defined in the units mapping settings.

See also

Using unit mapping (Page 81)

2.5.11 ZONE mapping and SITE mapping

The SITE mapping and ZONE mapping is evaluated when you call the command "Create" or "Create hierarchy" at an object in the working area. You use ZONE mapping and SITE mapping to specify under which SITE and ZONE objects are to be created in PDMS/E3D. The objects in the "3D work area" are sorted according to the mapping.

The mapping is defined in two tables at the "Folders for class/subclass definitions (PDMS)" object:

- The SITE mapping supplies the PDMS name of a SITE
- The ZONE mapping supplies the PDMS name of a ZONE

When the specified SITE cannot be found, the object is not created.

When the specified ZONE cannot be found, the ZONE is created and the object is created.

Condition

SITE mapping and ZONE mapping is performed only in the following case:

- SITE mapping and ZONE mapping are activated. See also chapter Using ZONE mapping and SITE mapping (Page 82).
- The interface operation "Create" or "Create hierarchy" is called.
- The subclass definition object of an interface object stipulates that the object has a ZONE as its owner in PDMS/E3D.

2.5.12 Operation messages

Definition

Short operation messages are available for the following interface operations:

- "Assign"
- "Export" from PDMS and COMOS
- "Refresh"
- "Unassign"

If you activate a history log for these operations, a window opens once the interface operation is complete containing information regarding the history of the operation for the user.

See also

Activating operation messages (Page 78)

2.5.13 Connection information

Definition

A COMOS interface object and a PDMS interface object are connected when their connection information matches.

COMOS object		PDMS object
"SystemUID"	=	":ComosUID"
"Y00T00012.Y00A01244" "PDMS reference number"	=	"RefNo"

COMOS object		COMOS object
"SystemUID"	=	"Y00T00012.Y00A00730" "SystemUID"

PDMS object		PDMS object
"RefNo"	=	":ComosCRefNo"
":ComosUID"	=	"SystemUID"

These properties are set during the following interface operations:

- PDMS and COMOS: "Export"
- COMOS: "Export to CE"
- COMOS: "Assign"

Name matching

If the name mapping of a COMOS object generates a name that matches the name of a PDMS object, the two objects are regarded as connected.

Name matching is the assignment criterion for the "Assign > Match Names" interface operation.

2.6 Configuring interface operations

The following approaches are possible for the administration of the interface operations area:

Task-based administration

Use task-based administration if you do not yet have extensive experience in administering the interface.

1. You start by only making the settings which are necessary for a specific task.
2. Then you check your entries by executing the corresponding interface operations.
3. After this you configure the next task, and so on.

Object-based administration

Use object-based administration if you have extensive experience in administering the interface.

1. Create the "Folders for class/subclass definitions" object and, below it, the class definition objects and the subclass definition objects. See also chapter Classes and subclasses (Page 58).
   The interface objects are or have been created by users during project planning.
2. You then configure the objects in the following sequence:
   – "Y00R00033 @PDMSMAP Folder for class/subclass definitions (PDMS)" object
   – Subclass definition objects
   – Base objects of the interface objects

Note

Configuration of the attributes

The properties of the attributes listed in the following have already been fully configured in the database.

Only change the preconfigured settings if the documentation explicitly indicates that such a change is permitted or necessary.

In particular:

- Do not change or modify any scripts.
- Do not modify or delete any links.
- Do not delete standard table assignments.

See also

Attributes of the "Folders for class/subclass definitions" object (Page 131)

Attributes of the subclass definition objects of classes "Cable" and "Tagged item" (Page 133)

Attributes of the subclass definition objects of the "Query" class (Page 136)

Attributes of the subclass definition objects of the "Document" class (Page 137)

Attributes of the interface objects (Page 138)

Attributes of the design template objects (Page 140)

2.6.1 Task-based administration

Task-based administration of the interface operations is described in the following:

1. Basic configuration:
   Configuration of the "Folders for class/subclass definitions" object, the subclass definition objects of the "Cable" and "Tagged item" classes, and the interface objects.
   See also chapter Performing the basic configuration (Page 73).
2. Use of COMOS queries in PDMS
   See also chapter Configuring the use of COMOS queries in PDMS (Page 75).
3. Use of pre/post executables
   See also chapter Configuring the use of pre/post executables (Page 75).
4. Importing DocLinks
   See also chapter Configuring the import of DocLinks (Page 76).
5. Importing AVEVA templates
   See also chapter Configuring the import of AVEVA templates (Page 76).

2.6.1.1 Performing the basic configuration

Procedure

1. Insert the "External 3D interface" tab for the base objects of the interface objects.
2. Optional: Enter additional element types in the "PDMS element types" standard table.
3. Create the "Folders for class/subclass definitions" object below the root node.
4. Optional: To configure the use of pre/post executables, go to the following chapter Configuring the use of pre/post executables (Page 75).
5. Create the necessary class definition objects.
6. Create the necessary subclass definition objects below the class definition objects.
7. Optional: To configure the import of AVEVA templates, go to the following chapter: Configuring the import of AVEVA templates (Page 76).

8. Edit the subclass definition objects of classes "Cable" and "Tagged item":
   - Define the structural behavior.
   - Define the PDMS element types.
   - Define the restriction rules for owners by selecting the element type of the owner. Optional: Define additional restrictions.
   - Define the site mapping and location mapping.
   - Define the model information.
   - In the name directory, define how values are mapped to one another in COMOS and PDMS.

   Optional:
   - Define local variables in the name directory.
   - Define the level rules.
9. Optional: To configure the use of COMOS queries in PDMS, go to the following chapter Configuring the use of COMOS queries in PDMS (Page 75).
10. Optional: Configure the "External 3D interface" tab.
11. Edit the "Folders for class/subclass definitions" object.
    - Define the character mapping.
    - If the "Export" operation is to be used: Define the site mapping and zone mapping.
    - Optional: Define the global variables.
    - Optional: Activate the history logs.
    - Define the unit mapping.
12. Optional: To configure the import of DocLinks, go to the following chapter: Configuring the import of DocLinks (Page 76).
13. Synchronize the settings between COMOS and PDMS.
14. Test the settings:
    In COMOS:
    Select "PDMS > 3D-View > Select > By Name":
    - If the object is found in PDMS: Run "PDMS > Assign> Selected object".
    - If the object is not found in PDMS: Run "PDMS > Export" and check the generated name in PDMS.

    In PDMS:
    Select "Comos > CE > Navigate".
15. Open the "General" tab for the subclass definition objects of classes "Cable" and "Tagged item".
    Specify the base objects for creating objects.
16. Synchronize the settings between COMOS and PDMS.

17. Use the following interface operations to test the settings in COMOS:
    - "Refresh"
    - "Custom Refresh"
    - "Check status"
    - "Export" (with attribute values)
18. Synchronize the settings between COMOS and PDMS.
19. Use the "Export" interface operation to test the settings in PDMS.

2.6.1.2 Configuring the use of COMOS queries in PDMS

Requirement

You have performed steps 1 - 6 as part of the basic configuration.

See also chapter Performing the basic configuration (Page 73).

Procedure

1. Create and configure the main queries that are to be called from within PDMS.
   Optional: Create and configure start queries for the main queries.
2. Configure the subclass definition objects of the "Query" class.
3. Synchronize the settings between COMOS and PDMS for the "Folders for class/subclass definitions" object.
4. Test the settings in PDMS in the "Query Comos data" window.

2.6.1.3 Configuring the use of pre/post executables

Requirement

You have performed steps 1 - 3 as part of the basic configuration.

See also chapter Performing the basic configuration (Page 73).

Procedure

1. Declare and implement PML functions.
2. Register the PML functions as pre/post executables for the "Folders for class/subclass definitions" object.
3. Synchronize the settings between COMOS and PDMS.
4. Test the settings by executing the interface operation for which you registered a PML function as pre/post executables.

2.6.1.4 Configuring the import of DocLinks

Requirement

You have performed steps 1 - 11 as part of the basic configuration.

See also chapter Performing the basic configuration (Page 73).

Procedure

1. Define the attribute or the UDA for the name of the COMOS document in the draft structure in PDMS.
2. Configure the subclass definition objects of the "Document" class.
3. Test the settings:
   - Create a document in COMOS.
   - Enter the name of the document in PDMS in the attribute defined at the beginning or the UDA.
   - Execute the "Import DocLinks" operation in PDMS.

2.6.1.5 Configuring the import of AVEVA templates

Requirement

You have performed steps 1 - 6 as part of the basic configuration.

See also chapter Performing the basic configuration (Page 73).

Procedure

1. Navigate to the following base object: "@20 > D30 > A30 > Y00R00038 @TPL Objects for design template structure".
2. Call the "Update design templates" command from the context menu of the node.
   GType folders are created in the base data.
3. Assign references to the device catalog to the created GType folder.

2.6.2 Maintaining the standard table for PDMS element types

Purpose

The following standard table is supplied with the database:

"Standard tables > @40 > Y00 > A10 > C10 > Y00N00465 PDMS element types"

The most important PDMS element types are listed in the standard table.

You can add the following to the standard table:

- Missing PDMS element types
- PDMS Version 12 and higher: User-defined PDMS element types

See also

Structure of the standard table for PDMS element types (Page 129)

2.6.3 Standard tables for classes and subclasses

System-internal management

The standard tables in which classes and subclasses are listed are managed system-internally in the engineering projects. This means that you are not permitted to edit the standard tables in the engineering project.

You may edit the standard table in the base project.

Standard table for classes

You find the standard table in the following node:

"Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A01 Classes"

Standard table for subclasses

You find the standard table in the following node:

"Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A02 Subclasses"

The standard table is dynamically updated: When you call the "PDMS > Save subclasses" context command at the "Folders for class/subclass definitions" folder, COMOS synchronizes the structure of the subclass definition objects underneath the class definition objects with the entries of the standard table.

See also

Structure of the standard table for classes (Page 130)

Structure of the standard table for subclasses (Page 130)

2.6.4 Creating the "Folders for class/subclass definitions" object

In the database supplied, you create the object directly underneath the project root.

Procedure

Create the "Folders for class/subclass definitions" object via the "New" entry in the context menu.

Note

If you create the folder using drag&drop from the base data, enter the following value in the "Name" property: "Y00R00033"

See also

Configuring the "Folders for class/subclass definitions" object (Page 78)

"Folders for class/subclass definitions" object (Page 62)

2.6.5 Configuring the "Folders for class/subclass definitions" object

2.6.5.1 Activating operation messages

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

To activate/deactivate operation messages, proceed as follows:

1. Open the properties of the "Folders for class/subclass definitions" object.
2. Select the "Attributes > General" tab.
3. Search for the "Operation messages" control group.
   The control group contains a table with the following columns:
   – "Operation" column: Interface operations for which operation messages are available
   – "Show information" column: To activate/deactivate operation messages
4. Select an entry from the lists in the "Show information" column.
   Possible values:
   – "Yes": Activates operation messages for the corresponding operation based on the row you have selected.
   – "No": Operation messages are deactivated for this operation.

See also

Operation messages (Page 70)

2.6.5.2 Defining global variables

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

1. Navigate to the "Folders for class/subclass definitions" object
2. Select the "Name directory" tab.
3. When defining global variables, proceed in exactly the same way as when defining a local variable in a String parameter for the subclass definition objects. See also chapter Working with String Parameters in the name directory (Page 93).

Structure of the "Name directory" table

The global variables are defined in the "Name directory" table. The table structure is virtually identical to the one for the subclass definition objects.

Differences:

- No COMOS unit
- No SetFunction
- Only one flag: "Rule"

Since only global variables are defined in the name directory of the "Folders for class/subclass definitions" object, there is no SetFunction and only the "Rule" flag.

Using the "Rule" flag

If you activate the "Rule" flag, you have to configure the String parameter in such a way that it returns both a COMOS value and a PDMS value.

Result: The interface compares the two values before every interface operation and only executes the operation if the values are identical.

See also

"Name directory" tab (Page 132)

2.6.5.3 Using pre/post executables

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

To execute a PML function before an interface operation is executed or after it has been completed, proceed as follows:

1. Declare the PML function.
   - Signature:
     If "Uses query" = "Yes"
     `!!NamePMLFunktion(!InterfaceOp is String, !config is Boolean, !arg1 is String, !arg2 is String, ...)`
     If "Uses query" = "No"
     `!!NamePMLFunktion(!InterfaceOp is String, !arg1 is String, !arg2 is String, ...)`
   - Parameter `InterfaceOp`: The name of the interface operation before or after which the PML function is executed.
   - Parameter `!config`: Required only if you select the value "Yes" in the "Before call" or "After call" table in the "Uses query" column.
   - Parameter `arg1`, `arg2`, ...: Name of any number of global String parameters: optional parameter
2. Implement the PML function.
3. Open the properties of the "Folders for class/subclass definitions" object.
4. Switch to the "Attributes > Pre/Post executables" tab.
5. If the PML function is executed before the start of the interface operation, register it in the "Pre executables" table.
   If the PML function is executed after completion of the interface operation, register it in the "Post executables" table.
   To register a PML function for an interface operation, proceed as follows:
   - "Executable" column:
     Enter the name of the PML function from step 1.
   - "Query" column:
     Select "Yes" if the PML function is to execute a COMOS query. The function must have been implemented accordingly.
     The PML function is then called twice. Initially with `!config = TRUE`:
     Implement the function in such a way that the config file for the query is configured in the function and sent to COMOS.
     COMOS then executes the query and the function retrieves the results.
     For the second time with `!config = FALSE`:
     Function operates with results
     Select "No" if the PML function does not use a COMOS query.
   - "Export", "Assign", "Refresh", "Check status", "Import ", "MTO import" columns:
     Select "Yes" to register the PML function for these interface operations.

You can find additional information on this topic in the "3D Integration Operation" manual, keyword "List of interface operations and internal names".

See also

"Pre/post executables" tab (Page 132)

Pre/post executables (Page 69)

2.6.5.4 Using character mapping

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

To define those characters which are permitted in COMOS but not in PDMS and are replaced during character mapping, proceed as follows:

1. Open the properties of the "Folders for class/subclass definitions" object.
2. Select the "Attributes > Character mapping" tab.
3. Configure the table which you find on this tab.
   For each line, define an illegal character along with the character which is to replace the illegal character in PDMS. Proceed as follows:
   – "Comos" column: Enter the illegal character.
   – "PDMS" column: Enter the character which is to replace the illegal character.

   Enclose the characters in single quotation marks.

See also

"Character mapping" tab (Page 132)

Character mapping (Page 69)

2.6.5.5 Using unit mapping

With the unit mapping, you define how units in PDMS and COMOS are mapped to one another if a different unit has been assigned to an attribute in COMOS than in PDMS.

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

1. Open the properties of the "@PDMSMAP Folders for class/subclass definitions (PDMS)" object.
2. Select the "Attributes > Unit mapping" tab.
3. Configure the table which you find on this tab:
   - "Name" column: The name of the mapping entry
     The name is displayed in the name directory in the "Unit" column.
   - "Unit in PDMS" column: No longer used.
   - "Unit group in COMOS" column: The unit group to which the unit entered in "Unit in COMOS" belongs.
     The unit group establishes the connection between the unit at the COMOS attribute and unit, which is specified in PDMS or expected during an exporting to PDMS.
   - "Unit in Comos" column:
     With export to PDMS: The unit into which the values registered in COMOS attributes are to be converted. This is the unit that is expected in PDMS.
     With import from PDMS: The unit of imported PDMS values, which is to be converted into the unit of a COMOS attribute.
     Note: During the import/export there is no explicit check whether the attribute actually uses the unit specified in the unit mapping in PDMS. The interface assumes that the correct unit has been entered.

Result

The mapping entries defined here are available in the name directory.

- During import:
  The value from PDMS is converted to the unit that exists for the COMOS attribute.
- During export:
  The value from COMOS is converted to the unit, that is specified in the "Unit in COMOS" column.

See also

"Unit mapping" tab (Page 133)

Units mapping (Page 69)

2.6.5.6 Using ZONE mapping and SITE mapping

Requirement

You have created the "Folders for class/subclass definitions" object in the engineering data. See also chapter Creating the "Folders for class/subclass definitions" object (Page 77).

Procedure

1. Open the properties of the "Folders for class/subclass definitions" object.
2. Select the "Attributes > General" tab.
3. Enable the "Use ZONE and SITE mapping" option in the "General properties" control group.

   Note

   Deactivation locks the "Export" menu

   If you deactivate the option and the subclass of an interface object prescribes that the object is located under a site or a zone in PDMS, the "PDMS > Export" command will be locked in the context menu of the COMOS interface object. You cannot create any SITES or ZONES.

4. SITE mapping: Switch to the "SITE mapping" tab.
5. ZONE mapping: Switch to the "ZONE mapping" tab.
6. Configure the mapping table. The configuration is similar to that of the name mapping for the subclass definition object.
   Differences:
   - Configure the table in such a way that the mapping entries generate the name of a SITE or ZONE.
   - Mapping is only performed for one direction: It generates the PDMS name of a SITE or ZONE.
   - There are no level rules.
   - The structural behavior has no influence.

Result

If a user requests COMOS objects in the COMOS PDMS Interface, names for PDMS SITES and ZONES are created for cables and devices based on the defined rules. The SITES and ZONES are created in COMOS PDMS Interface and the COMOS objects are sorted underneath them.

Note

Display in the "COMOS objects" area depends on mapping

If you do not configure SITE mapping and ZONE mapping, all objects are stored parallel to one another in the "COMOS objects" area of the PDMS/E3D Engineering Interface.

If you have only configured one of the two tables or the configurations cannot be evaluated, the objects are created under a "Not categorized" node.

See also

"Site mapping" and "Zone mapping" tabs (Page 133)

Configuring the generation of a COMOS object path (Page 108)

ZONE mapping and SITE mapping (Page 70)

2.6.6 Creating a class definition object

You can only create one class definition object for each class.

Procedure

To create a class definition object, proceed as follows:

1. Click the "Units" tab in the Navigator.
2. Select the "Folders for class/subclass definitions" object in the Navigator.
3. Select the "New" command from the context menu and select the required class.

If you have already created a class definition object for the class, the class will no longer be available in the context menu.

Defining the properties of the class

You define the subclass for the class by creating subclass definition objects under a class definition object.

Name of the class definition object

You recognize the mapping of a class definition object to a class based on its name.

When creating the object, COMOS automatically enters a name - depending on which class you selected under "New" in the context menu.

Do not change the name of the class definition object.

See also

Classes and subclasses (Page 58)

2.6.7 Creating subclass definition objects

Procedure

To create a subclass definition object, proceed as follows:

1. Click the "Units" tab in the Navigator.
2. Open the "Folders for class/subclass definitions" object in the Navigator.
3. Select the class definition object under which you want to create a subclass definition object.
4. Select the "New > <Subclass definition object>" command from the context menu.
5. Open the properties of the subclass definition object.

6. In the "Name" property, enter the name of the subclass which describes the objects that belong to the subclass. See also chapter Structure of the standard table for subclasses (Page 130).
7. Enter a description in the "Description" property.

Result

- The subclass definition object is created.
- The subclass definition object is automatically assigned the class that is entered as the name of its class definition object.

Adding a subclass to a standard table

Subclasses can be added to the standard table for subclasses. See also chapter Standard tables for classes and subclasses (Page 77).

2.6.8 Configure subclass definition objects of classes "Cable" and "Tagged item"

2.6.8.1 Defining PDMS element types

Procedure

To define which PDMS element types are permitted for interface objects of this subclass, proceed as follows:

1. Open the properties of the subclass definition object.
2. Switch to the "Attributes > General" tab.
3. Click on a field in the "Name" column in the "PDMS element types" table.
   A list is displayed in the field.
4. Open the list and select the required element type from the list.
   The type is entered in the field. COMOS automatically adds a new line to the bottom of the table.
5. Repeat step 3 until you have entered all required element types.

Result

Only PDMS element types defined in this way are available to the user in the interface objects of this subclass.

2.6.8.2 Defining structural behavior

Procedure

To define the structural behavior of an interface object of this subclass, proceed as follows:

1. Open the properties of the subclass definition object.
2. Switch to the "Attributes > General" tab.
3. In the "Structural behavior" control group, "Creation mode" list, define whether the COMOS object connected to the PDMS interface object is located in the unit tree or in the location tree.
   Possible values:
   – "Create in unit tree": Site mapping is performed to find or create the COMOS interface object.
   – "Creation in location tree": Location mapping is performed to find or create the COMOS interface object.
   – "None": You can find information on this in the reference part of this document, in chapter "General" tab (Page 131).
4. "Structural behavior" control group, "Assign mode" list:
   Define whether the COMOS interface object references another COMOS object and if the referenced object is located in the unit tree or in the location tree.
   The values that are offered to you are determined by which "Creation mode" you have set.
   Possible values in the "Assignment mode" list:
   – "Assign object in the unit tree": The COMOS interface object gets a unit reference. The referenced object is determined by means of site mapping.
   – "Assign location in the location tree": The COMOS interface object gets a location reference. The referenced object is determined by means of location mapping.
   – "None": The interface object gets no reference.

See also

Working with area pipes (Page 65)

2.6.8.3 Defining the base object for the creation of interface objects

Possible alternatives

There are three ways of defining which base object is used to create an interface object in COMOS:

1. Specify the base object on the "General" tab of the corresponding subclass definition object.
2. Specify the base object by mapping the name of the corresponding subclass definition object.
   This is the method used in this document.
3. Enter the SystemFullName of the base object in the following UDA ":ComosBaseOb" of the PDMS interface object.
   This method offers you the most flexibility but it is also the most complex, since the UDA is set manually for each PDMS interface object.

Procedure

To define which base object is used to create the interface objects of this subclass in COMOS, proceed as follows:

1. If PDMS interface objects of this subclass already exist: Make sure that the ":ComosBaseOb" UDA of the PDMS interface object is empty. If not, the interface will use the base object that is entered in the UDA.
2. Open the properties of the subclass definition object.
3. Go to the "Attributes > Name mapping" tab.
   Make sure that no base object has been defined for the mapping entry on the far right in the site mapping or location mapping.
4. Switch to the "Attributes > General" tab.
5. "Base object for creating objects" control group, "Create in unit tree" reference:
   In order to create the interface object in COMOS, you define which base object will be used during the site mapping with a drag-and-drop operation.
   If one of the following values is set in the "Structural behavior" control group, set a base object reference:
   - Creation mode: "Create in unit tree"
   - Assign mode: "Assign to object in unit tree"
6. "Base object for creating objects" control group, "Create in location tree" reference:
   Using drag&drop, specify which base object is used during location mapping to create the interface object in COMOS.
   Has to be set if one of the following values is set in the "Structural behavior" control group:
   - Creation mode: "Create in location tree"
   - Assign mode: "Assign to location in location tree"

See also

Algorithm 1 for generating a COMOS object path (Page 106)

2.6.8.4 Automatically inserting additional hierarchy levels

You can specify that a hierarchy level will be added in the "COMOS objects" area of the PDMS/E3D Engineering Interface for COMOS objects that are assigned to a specific subclass and have a two-level structure.

Note

The functionality only works if the top object of the three-level structure in the PDMS/E3D Engineering Interface is located below a ZONE.

Top-level objects are objects that can be created directly below ZONEs.

You can use this function for the following objects:

- Equipment with nozzles, such as vessels
  When you request equipment with nozzles from COMOS in the interface, you can specify that a three-stage structure is automatically created in the interface.
  Example:

COMOS	PDMS/E3D Engineering Interface
Containers	Containers PDMS type: EQUI
	SUBE The SUBE is created by the "Owner subclass" function.
Nozzles	Nozzles PDMS type: NOZZ

- When working with KKS: for pipe parts, such as valves
  When you are working with the KKS labeling system and request pipe parts in the interface which are included in a two-stage structure in COMOS, you can specify that a three-stage structure is automatically created in the interface.
  Example:

COMOS	PDMS/E3D Engineering Interface
Pipe branch	PIPE
	Pipe branch PDMS type: BRAN The pipe branch is created by the "Owner subclass" function.
Valve	Valve PDMS type: VALV

Procedure

1. Open the properties of a subclass for nozzles or for pipe parts.
2. Open the "Attributes > General" tab.

3. For nozzles:
   In the "Owner subclass" list, select a subclass that has a "SUBE - Sub equipment" entry as "PDMS element type". Make sure that the element type of all objects of the structure involved adhere to the PDMS rules.
   For pipe parts:
   In the "Owner subclass" list, select a subclass that has a "BRAN - Branch" entry as "PDMS element type". Make sure that the element type of all objects of the structure involved adhere to the PDMS rules.
4. Open the "Attributes > Owner restriction rules" tab.
5. For nozzles:
   Select the "SUBE - Sub equipment" entry from the "Element type of owner" list.
   For pipe parts:
   Select the "BRAN - Branch" entry from the "Element type of owner" list.

Result

When objects listed above that are contained in a two-level structure in COMOS are requested in the interface, a three-level structure is automatically created in the PDMS/E3D Engineering Interface. An object is created for the object that is missing in the COMOS structure; this object has the PDMS element type "SUBE" or "BRAN".

You can create virtual objects and the underlying hierarchies in PDMS. You can find additional information on this topic in the "3D Integration Operation" manual, keyword "Creating objects in PDMS".

2.6.8.5 Specifying restriction rules for owners

Requirement

You are familiar with the objective of the restriction rules for owners. See also chapter Restriction rules for owners (Page 67).

Rule types

The following rule types exist:

- The PDMS element type that the owner must have in PDMS.
- An expression which is evaluated by the owner and must be true.
- A filter which checks whether a given owner attribute contains a specific occur string.

Defining PDMS element types

1. Open the properties of a subclass definition object.
2. Go to the "Attributes > Owner restriction rules" tab.

3. In the "Element type of owner" list, select the PDMS element type that the owner of the interface object must have in PDMS.
4. Optional: In the "Conditions" control group, define any number of expressions and/or filters which are applied to the potential owner. See section "Defining filters".

Result

When the potential owner conforms to the rules, the object can be created below the owner.

Defining expressions

1. In the top table in the "Conditions" control group, click in the "Expression" column.
2. Enter an expression for which the following applies:
   – it is compatible with a PDMS block object
   – it returns a boolean type value
3. Press <Enter> to confirm your entry.

Result

When the potential owner conforms to the rules, the object can be created below the owner.

- The expression is evaluated in PDMS at the potential owner.
  – Return value "True": The owner meets the condition.
  – Return value "False": The owner does not meet the condition.
- A line for another expression is added to the table.

Example:

```
name.Substring(1,2).eq('EP')
```

Defining filters

1. In the bottom table in the "Conditions" control group, click in the "Attribute" column.
2. Enter the name of the attribute as it is defined in PDMS.
3. Click in the "Occur string" column.
4. Enter a value which is to be evaluated at the attribute.
   You can enter the following values:
   – String
   – Local or global variable
     Exception: Do not use local variables which get their value from PDMS.
5. Press <ENTER> to confirm your entry.

Result

The filter is evaluated in PDMS at the potential owner.

- If you have entered a string: The object can be created below the owner if the string is a substring of the attribute.
- If you have entered a variable: The object can be created below the owner if the value of the variable is a substring of the attribute.

2.6.8.6 Configuring the model

Requirement

If you are using design templates, the design templates must be imported from PDMS.

You can find information about importing design templates in chapter Importing AVEVA design templates (Page 122).

Inheritance of model information to interface objects

The database is configured in such a way that the interface objects apply the model information from their subclass.

If the interface objects are to use different model information from what is specified by their subclass: Define different model information in the base objects of the interface objects.

Users can overwrite the settings taken from the subclass or the base objects in the engineering view.

Procedure

To define model information at the subclass definition object, proceed as follows:

1. Open the properties of the subclass definition object and switch to the "Attributes > Model" tab.
2. If the geometry is based on a design template: Enter the GTypes in the "GTypes for filtering" field. Define the GTypes that are available for selection in design templates.
   If you enter multiple GTypes, use ";" as a separator.

3. In the "Mode" list, define how the model is generated. Select one of the following modes:

Mode	Subclass of "Cable" class	Subclass of "Tagged item" class	Result
"Design template"	/	Available for subclasses with the "EQUI" PDMS element type	The geometry is based on a design template.
"PDMS element"	/	Available	The geometry is copied from an existing PDMS element.
"Function"	Available	Available	The geometry is generated by a function which is also performs other initializations.
"None"	Available	Available	A box is generated.

4. You define the details for the generation of the model based on which mode you have selected. See also chapter Overwriting the inherited model information (Page 124).

See also

"Model" tab (Page 135)

2.6.9 Configure 'MTO (PDMS)' class

Procedure

1. Create the "Folder for class/subclass definitions (PDMS)" object.
   You can find more information on this topic in the manual "3D Integration Administration", keyword "Create folder for classes / subclasses definitions" object".
2. Use the context menu to create the "Class - MTO (PDMS)" object below this.
3. Open the "Attributes > General" tab in the properties of the object.
4. From the "Subclass for generating pipelines" list, select a subclass to be used for generating pipelines after the import.
5. Optional: Select another object in the "Base object for MTO object" field.
   This base object is used to generate imported MTO objects.
6. Optional: Select another object in the "Base object for bolts" field.
   This base object is used to generate imported bolts.
7. Optional: In the "Bolting" table, change the attributes in which the properties of imported objects are to be stored.

8. Optional: Configure the import of pipe lengths.
   Activate the option "Import pipe length".
   Enter the NestedName of the COMOS attribute you want to use in the "Pipe length attribute" field. The representation type has to be “List” in the properties of the attribute.
9. Select the "Name directory" tab.
   Configure the table.
   See also chapter Working with String Parameters in the name directory (Page 93).

2.6.10 Working with String Parameters in the name directory

2.6.10.1 Name directory

Definition

The name directory is a table in which each line defines a string parameter.

See also chapter String Parameters (Page 66).

Evaluation

The name directory is evaluated in the following cases:

- During an interface operation if values are compared, exported or imported. These are usually attribute values.
- If a string parameter is used in a subclass definition object on the "Name mapping" tab to generate one of the following components of a PDMS name part:
  - Prefix
  - Postfix
  - Data item
  - Base object
- If objects are filtered in the "Export to Comos" PDMS mask.
- If a string parameter is transferred to a PML function as a parameter. For example, if a String parameter is used in an initialization function in a subclass definition object on the "Model" tab.

2.6.10.2 Tasks of the String parameter:

The tasks marked with "*" provide the basis for the remaining tasks.

A String parameter can fulfill the following tasks:

- Assign a COMOS value to a variable*
- Assign a PDMS value to a variable*
- Write a PDMS value to a COMOS attribute (import)

- Write a COMOS value to a PDMS object (export)
- Generate a component of a mapping entry in the name mapping (variable definition)
- An interface operation compares the value of a COMOS interface object returned by the String parameter with the value of the connected PDMS object returned by the String parameter (check).
- The "Export" interface operation uses the String parameters to check whether it is permissible to create an object below a specific owner.

2.6.10.3 Structure of a String Parameter

String parameters are defined in the mapping table on the "Name directory" tab. Each line defines a String parameter.

A String parameter has the following constituent parts:

Constituent	Description
Name	Displayed in the interface for identification.
Value of the String data type	The value is calculated/set by evaluating one of the following columns: • "COMOS attribute" • "PDMS attribute/expression" • "GetFunction" • "SetFunction" See also section "Value of String data type" below.
Flags which define how the String parameter is processed by interface operations	The flags are set in the following columns: • "Check status" • "Export" • "Import" • "Admin" • "Rule"
Entry from the unit mapping of the "Folders for class/subclass definitions" object	Defines the unit that is expected by or will be transferred by COMOS and PDMS. Values can be converted as appropriate.
Data type transferred to the PLM function in PDMS and into which the string of the string parameter is converted in PDMS	If no PLM function is used, the type of the PDMS attribute is used.

Value of String data type

A string involves one of the following values:

Value comes from COMOS	Value comes from PDMS
Attribute value of the interface object that is currently being processed by the interface operation that is evaluating the name directory.	Attribute value of the PDMS Interface object that is set as the current element.
Any other property of an object other than the interface object, such as the name or the label.	Value that is returned by a PDMS expression. The PDMS expression is evaluated once the currently examined PDMS Interface object has been set as the current element.
Fixed string	Fixed string
Special case: Attribute value of an object other than the interface object.	/

2.6.10.4 Configuring String Parameters

Procedure

1. Open the properties of the subclass definition object.
2. Switch to the "Attributes > Name directory" tab.
3. Configure the "Name directory" table:
   – Enter the name used to address the String parameter in the "Name" column.
   – Configure the other columns of the String parameter depending on which task the String parameter is to fulfill. The individual configuration options can be combined so that a String parameter fulfils different tasks.

2.6.10.5 Assigning a COMOS value to a variable

Procedure

1. Configure the "Comos attribute" column.
   See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
   Permitted values:
   – Attribute name
   – Fixed string
   – Call of an owner method
2. Select any data type in the "PDMS type" column.

2.6.10.6 Assigning a PDMS value to a variable.

Procedure

1. Configure the "PDMS attribute/expression" or "GetFunction" column. See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).

   Note

   When the "GetFunction" and "PDMS attribute/expression" columns are set, GetFunction has a higher priority.

2. Select a data type in the "PDMS type" column.

2.6.10.7 Writing a PDMS value to a COMOS attribute (import)

Procedure

1. Enter an attribute name in the "Comos attribute" column. See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
2. Configure the "PDMS attribute/expression" or "GetFunction" column. See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).

   Note

   When the "GetFunction" and "PDMS attribute/expression" columns are set, GetFunction has a higher priority.

3. Select a suitable entry in the "Unit" column.
   You are offered the entries from the unit mapping in the "Folders for class/subclass definitions" folder.
4. Specify the data type that is to be transferred to the PML function in the "PDMS type" column.
5. Select the "Yes" entry in the "Import" column.

   Note

   If the "Export" and "Import" columns are activated, the "Export" column has a higher priority. The "Import" column is not evaluated.

2.6.10.8 Writing a COMOS value to a PDMS object (export)

Procedure

1. Configure the "Comos attribute" column.
   Permitted values:
   - Attribute name
   - Fixed string
   - Call of an owner method

   See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
2. Configure the "SetFunction" or "PDMS attribute/expression" column: See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).

   Note

   When the "SetFunction" and "PDMS attribute/expression" columns are set, SetFunction has a higher priority.

3. Select a suitable entry in the "Unit" column.
   You are offered the entries from the unit mapping in the "Folders for class/subclass definitions" folder.
4. Specify the data type that is to be transferred to the PML function in the "PDMS type" column.
5. Select the "Yes" entry in the "Export" column.

   Note

   If the "Export" and "Import" columns are activated, the "Export" column has a higher priority. The "Import" column is not evaluated.

2.6.10.9 Generating a component of the name mapping

Follow the procedure below so that the String parameter generates one of the following components of a mapping entry in the name mapping.

- "Data item"
- "Prefix"
- "Postfix"
- "Fixed name"

Procedure

Configure the following columns of the String parameter:

1. Select the "Yes" entry in the "Admin" column.
2. Define which string the String parameter returns in one of the following columns:
   - "Comos attribute"
   - "PDMS attribute/expression"
   - "GetFunction"

   If more than one column is set, following priority applies:
   "Comos attribute" > "GetFunction" > "PDMS attribute/expression"
   See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
3. Select a data type in the "PDMS type" column.

2.6.10.10 Configuring a value comparison

Follow the procedure below so that an interface operation compares the values of a COMOS object returned by the String parameter with the values of the PDMS object connected to it.

Procedure

1. Select the "Yes" entry in the "Check" column.
2. Access a COMOS value and a PDMS value by setting a value in the "Comos attribute" and "PDMS attribute/expression" columns or in the "GetFunction" column.

   Note

   When the "GetFunction" and "PDMS attribute/expression" columns are set, GetFunction has a higher priority.

   See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
3. Select a data type in the "PDMS type" column.

2.6.10.11 Checking an owner

Follow the procedure below so that the "Export" interface operation uses the String parameters to check whether it is permissible to create an object below a specific owner.

Procedure

1. Select the "Yes" entry in the "Rule" column.
2. Access a COMOS value and a PDMS value by setting a value in the "Comos attribute" and "PDMS attribute/expression" columns or in the "GetFunction" column.
   The PDMS expression GetFunction is called at the potential owner of the object that is to be created in PDMS.
   See also chapter Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns (Page 99).
3. Select a data type in the "PDMS type" column.
   The object is only created if the values are the same.

2.6.10.12 Configuring the "Comos attribute", "PDMS attribute/expression", "GetFunction", and "SetFunction" columns

Which of these columns you configure and which entries are permitted depends on the purpose for which you are using the String Parameter.

Permissible entries

Action	Column	Permissible entries
Get value from COMOS	Column: "Comos attribute":	• Attribute name • Fixed string • Call of an owner method
Set value in COMOS	Column: "Comos attribute":	• Attribute name
Get value from PDMS	"PDMS attribute/expression" column: or	• Attribute name • Expression • Fixed string
	"GetFunction" column:	• Name of a GetFunction
Set value in PDMS	Column: "PDMS attribute/expression": or	• Attribute name
	"SetFunction" column:	• Name of a SetFunction

Notation

Use the following notation:

Column	Notation
"Comos attribute"	• Attribute name: Use the NestedName: "<TabName>.<AttributeName>" The String Parameter returns the DisplayValue of the attribute. • Fixed string: Set the fixed string inside double quotation marks. • Call of an owner method: Enter an instruction in the field that outputs the desired value. Use the key word `Owner`. The same functions and properties are available as in the COMOS Object Debugger. Example: The String Parameter should return the label of the owner of the currently processed interface object: "Owner.Label"
"PDMS attribute/expression"	• Attribute name: Enter the name of the PDMS attribute. • Expression: Enter the PDMS expression. • Fixed string: Set the fixed string inside single quotation marks.
"GetFunction"	Enter the name of the GetFunction which is to be executed in PDMS. The GetFunction returns a PDMS value. Example: "!!GetPurpose()"
"SetFunction"	Enter the name of the SetFunction which is to be executed in PDMS. The SetFunction writes a value to PDMS. Example: "!!SetPurpose()"

See also

Configuring String Parameters (Page 95)

2.6.11 Configuring site mapping and location mapping in name mapping

2.6.11.1 Structure of the tables for unit mapping and location mapping

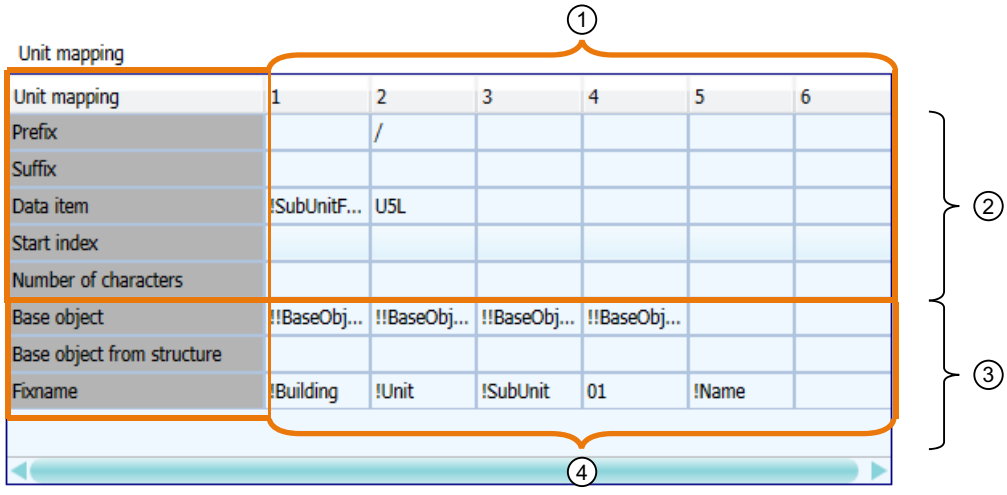
You will find the tables for unit mapping and location mapping at the subclass definition objects of the "Cable" and "Tagged item" classes in the "Name mapping" tab.

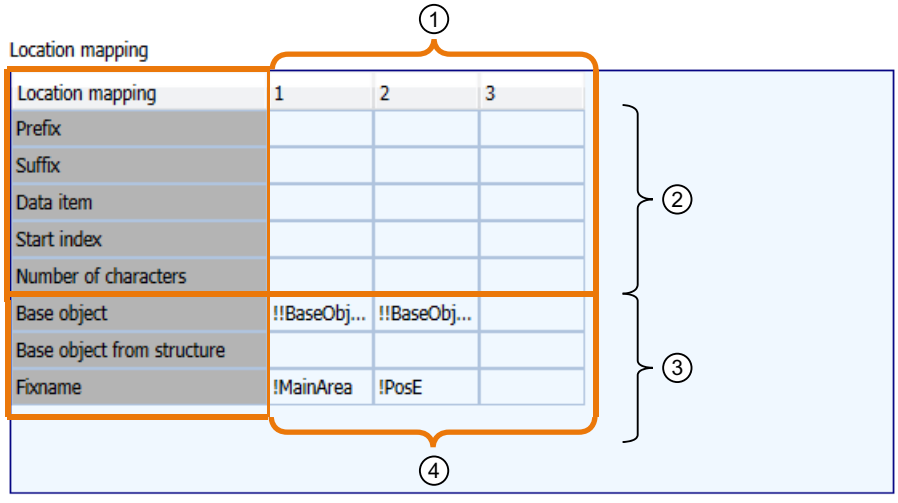
Each table has two purposes:

- Generate a PDMS/E3D name
- Generate a COMOS object path
  - Derive COMOS hierarchy and name of the COMOS object to be created.
  - Define base objects for generating the hierarchy and the COMOS object.
  - Assign a reference from one COMOS object to another COMOS object.

The tables are also used for the following operations in the Engineering Interface:

- "Assign by matching names" operation
- When updating the PDMS/E3D name: "Update names" operation





Number	Description
1	The columns show the name components of a PDMS name.
2	These rows are used for generating a PDMS/E3D name.

Number	Description
3	These rows are used for generating a COMOS object path and for creating the objects.
4	The columns show the hierarchy levels of the COMOS object path.

The mapping function is different in the plugins "COMOS PDMS Engineering Interface" and "COMOS PDMS Interface".

- See also chapter Mapping for the COMOS PDMS Interface (Page 102).
- See also chapter Mapping for the COMOS PDMS Engineering Interface (Page 109).

Use of unit mapping or location mapping

The creation mode set at the subclass determines which table is evaluated: "General" tab, "Structural behavior" control group, "Creation mode" list

- "Create in unit tree":
  Unit mapping is performed.
- "Create in location tree":
  Location mapping is performed.

Creation mode	Assignment mode	Usage
"Create in unit tree"	"Assign to location in location tree"	Unit mapping and location mapping
"Create in unit tree"	"None"	Unit mapping
"Create in location tree"	"Assign to location in unit tree"	Unit mapping and location mapping
"Create in location tree"	"None"	Location mapping

2.6.11.2 Mapping for the COMOS PDMS Interface

Algorithm for generating a PDMS name

A PDMS name comprises a number of name parts. Each column of the "Unit mapping" and "Location mapping" tables creates a name part. When put together, the entries produce a full PDMS name.

Elements of a PDMS name part

The PDMS name is defined by the following elements:

Row	Description
"Prefix"	Optional: The prefix of the PDMS name part.
"Suffix"	Optional: The suffix of the PDMS name part.
"Data item"	The data item of the PDMS name part.
"Start index"	The point where a name part is inserted in the full PDMS name.
"Number of characters"	The number of characters of the data item that are applied for the name part.

Algorithm

To generate the name of the PDMS object to which a COMOS object is to be connected, the interface proceeds as follows:

1. The interface checks which subclass the COMOS object has and evaluates the creation mode of the subclass definition object:
   - "Create in unit tree": Unit mapping is performed.
   - "Create in location tree": Location mapping is performed.
2. The columns are evaluated. The following algorithm is executed for each column:
   - The data item is evaluated.
   - If no data item was defined, the column is skipped. The algorithm continues with the next column.
   - If the data item is longer than is permitted by the "Number of characters" cell, it is truncated.
   - Prefix (if necessary truncated), data item and postfix are combined to form a character string. The character string represents a name part.
3. All PDMS name parts are combined to form a string which cannot be modified. Each name part is copied to the position in the string defined by its start index.
4. If the index ranges of two name parts overlap, the name part of the name part evaluated later overwrites the name part evaluated first.

Configuring generation of a PDMS name

Requirement

- You have defined the structural behavior.
- You are familiar with the algorithm for generating the PDMS name. See also chapter Algorithm for generating a PDMS name (Page 102).

Elements of a PDMS name part

Each PDMS name part is defined by the following elements:

Row	Description
"Prefix"	Prefix of the PDMS name part. • Optionally: • Any number of characters
"Suffix"	The suffix of the PDMS name part. • Optionally: • Any number of characters

Row	Description
"Data item"	The data item of the PDMS name part. Permitted items: • Composed expression which returns the name or label of an object from the unit tree or location tree • String parameter
"Start index"	The point where a name part is inserted in the full PDMS name. If nothing is entered, the name part is appended to the name parts composed up to this point.
"Number of characters"	The number of characters of the data item that are applied for the name part. • Number • Any number of characters permitted

Procedure

1. Open the properties of the subclass definition object of the "Cable" or "Tagged item" class.
2. Select the "Attributes > Name mapping" tab.
3. Identify where objects of the subclass are located or need to be created in your COMOS object structure. Split the overall path into its individual nodes.
4. Analyze how the PDMS name for objects of the subclass is to be structured. You decide how many name parts and therefore columns you are using.

5. Enter a data item in at least one column.
   The data item of a mapping entry contains either COMOS data or PDMS data. The following data is permitted:

COMOS data	PDMS data
• Return value of a String parameter which returns COMOS data.	Return value of a String parameter which returns PDMS data.
• An expression which returns the name or label of an object in the unit tree.	/
• An expression which returns the name or label of an object in the location tree.	/

   Notation of the data item:

Notation of the String parameter	Notation of the composed expression
"!<Name of a local String parameter>" example: "!Area"	First element: • In unit mapping: "U" • In location mapping: "L"
"!!<Name of a global String parameter>"	Second element: • "<Number>": Structure level in the tree on which the object is located In unit mapping: Starting from the project root In location mapping: Starting from the object "Folders for class/subclass definitions (PDMS)"
	Third element: • "N" or "L": Use the name or label of the object
	Example: "U2L": Use the label of an object on the second level underneath the project root in the unit tree.

6. Enter a start index and a number of characters in the same column.
7. Optional: Enter a prefix and a postfix.
8. Check which COMOS nodes and PDMS name parts can be mapped directly to one another.
   Configure a shared column for these nodes and name parts.
9. Check which COMOS node does not have a corresponding PDMS name part and vice versa.
   Configure a separate column for each of these nodes and name parts.
   Examples of COMOS nodes which do not have a corresponding PDMS name part:
   – Folder
   – The PDMS root object in the "Locations" tab
   – Other objects that are only used to structure the COMOS data
10. Configure the columns as described in the table below.

Algorithm 1 for generating a COMOS object path

Algorithm

- To create an object in COMOS and complete its owner structure, the base objects that are to be used must be known for each hierarchy level.
- To create the owner structure, COMOS evaluates the "Base object" or "Base object from structure" row.
- To create the object which is actually searched for, the interface evaluates the UDS ":ComosBaseOb" of the PDMS object.
  If the UDA is empty, the interface evaluates the "Base object for creating objects" control group in the "General" tab of the subclass definition object.
  If no base objects are specified in the "Base object for creating objects" control group, the interface evaluates the "Base object" or "Base object from structure" row in name mapping.

This documentation assumes that this setting is made in the "General" tab.

If you define the base object via name mapping, the algorithm described below is used.

Evaluation of the "Base object" row

The here specified base object is used.

Evaluation of the "Base object from structure" row

Only activate the "Base object from structure" setting if the project is linked to a project structure for KKS.

If the setting is activated, the interface proceeds as follows:

- It navigates to the project structure level specified in the current column.
- In a project structure, several objects with different base objects can be prepared for a single level. The interface compares the data item or fixed name of the column with the name mask of the possible base objects.
- The object is created with the base object whose name mask matches the data item or fixed name.
- The data item or fixed name is entered in the "Name" property of the new object.

See also

Defining the base object for the creation of interface objects (Page 87)

Algorithm 2 for generating a COMOS object path

Algorithm

1. The interface checks the subclass of the PDMS object.
2. The interface evaluates the creation mode of the corresponding subclass definition object: "General" tab, "Structural behavior" control group, "Creation mode" list:
   - "Create in unit tree": Unit mapping is performed.
   - "Create in location tree": Location mapping is performed.

The execution of the mapping tables involves the following steps:

1. The interface extracts the PDMS name parts from the PDMS name. To do this, it evaluates the start index and the number of characters of the corresponding mapping entry.
   For the last name part, the mapping entry on the far right is used; for the penultimate name part, the second mapping entry from the right is used, and so on.
2. The entire COMOS hierarchy of the object is composed according to this principle.

The following algorithm is executed for each mapping entry:

1. The interface checks what is entered in the "Data item" row:
   - Name of a String Parameter:
     Consequence: A corresponding COMOS node does not exist for the name part.
     Continue with step 2.
   - Name or label of a COMOS object:
     If there is one: The prefix and postfix are removed from the PDMS name part. The remaining string consists of the name or the label of a COMOS object. It is copied to the path list.
   - No entry: Continue with step 2.
2. If the "Data item" row is empty or contains a String parameter, a check is made as to whether the "Fixname" row has been configured:
   - "Fixname" is not set: The processing of the mapping table is aborted.
   - "Fixname" is set: The entered or resulting string is written to the path list.

Once all mapping entries have been processed, the sequence of the entries in the path list is reversed.

Result

- The string written to the path list produces the path to the COMOS object you are looking for.
- If the object does not exist yet, the mapping table returns all needed information in order to create it. The name part of the last mapping entry is written to the "Name" property of the COMOS object.
- If the interface detects that the owner structure of the object you are looking for is incomplete: The mapping table returns all information required to complete the owner structure.

See also

Algorithm 1 for generating a COMOS object path (Page 106)

Configuring the generation of a COMOS object path

The next chapter describes the generation of a COMOS object path according to algorithm 1.

Requirement

- You have defined the structural behavior.
- You are familiar with the algorithm for generating the COMOS object path. See also chapter Algorithm 1 for generating a COMOS object path (Page 106).

Elements for generating a COMOS path

Row	Function	Description
"Base object"	Base object used to create an object.	Used to complete the owner structure of the COMOS object or to create the object in COMOS. You only use one of the two rows.
"Base object from structure"	This row is only used for KKS structures. Specifies whether the base object from the KKS base data structure is to be used. • "Yes": If the project is linked to a project structure, the project structure is used for the generation of the COMOS objects. • "No": Even if a project structure is available and linked, it is not used.	
"Fixname"	Name of a COMOS object within the path to a COMOS object.	

Procedure

1. Open the properties of the subclass definition object of the "Cable" or "Tagged item" class.
2. Select the "Attributes > Name mapping" tab.
3. To define the base object of the first hierarchy level, enter the following in column "1":
4. When you are working in a KKS structure: In the "Base object from structure" row, select whether the project structure is to be used when generating the COMOS objects.
5. If you have selected "No" or are not working in a KKS structure: Enter which base object is to be used for generating the object.
   Possible values:
   – SystemFullName of a base object
     Use "\" to separate the individual nodes. Without quotation marks
   – Local String parameter: "!<Name of a local String parameter>"
   – Global String parameter: "!!<Name of a global String parameter>"

6. To define a name for the object that is to be created, enter a value in the "Fixname" column. Possible values:
   - Local String parameter: "!<Name of a local String parameter>"
   - Global String parameter: "!!<Name of a global String parameter>"
   - Fixed string: Without quotation marks
7. Repeat the steps for the remaining hierarchy levels.
   The column that is used last represents the object that is actually going to be created. This documentation assumes that the base object is defined in the "Base objects for creating objects" control group in the "General" tab of the subclass definition object. This means you do not define a base object in the mapping tables in the column that was used last.

Configuring level rules

Procedure

To define a deviation from the standard evaluation of site mapping and location mapping, proceed as follows:

1. Open the properties of the subclass definition object.
2. Activate the "Attributes > Name mapping" tab.
3. Right-click a field in the right-hand column of the "Level rules" table.
   Result: The field is activated and highlighted in color.
4. Right-click the field a second time with the mouse.
   Result: A list is displayed in the field.
5. To activate the rule, select "Yes".
   To deactivate the rule, select "No".

See also

Level rules of name mapping (Page 64)

2.6.11.3 Mapping for the COMOS PDMS Engineering Interface

Algorithm for generating a PDMS name

A PDMS name comprises a number of name parts. Each column of the "Unit mapping" and "Location mapping" tables creates a name part. When put together, the entries produce a full PDMS name.

Elements of a PDMS name part

The PDMS name is defined by the following elements:

Row	Description
"Prefix"	Optional: The prefix of the PDMS name part.
"Suffix"	Optional: The suffix of the PDMS name part.
"Data item"	The data item of the PDMS name part.
"Start index"	The point where a name part is inserted in the full PDMS name.
"Number of characters"	The number of characters of the data item that are applied for the name part.

Algorithm

To generate the name of the PDMS object to which a COMOS object is to be connected, the interface proceeds as follows:

1. The interface checks which subclass the COMOS object has and evaluates the creation mode of the subclass definition object:
   - "Create in unit tree": Unit mapping is performed.
   - "Create in location tree": Location mapping is performed.
2. The columns are evaluated. The following algorithm is executed for each column:
   - The data item is evaluated.
   - If no data item was defined, the column is skipped. The algorithm continues with the next column.
   - If the data item is longer than is permitted by the "Number of characters" cell, it is truncated.
   - Prefix (if necessary truncated), data item and postfix are combined to form a character string. The character string represents a name part.
3. All PDMS name parts are combined to form a string which cannot be modified. Each name part is copied to the position in the string defined by its start index.
4. If the index ranges of two name parts overlap, the name part of the name part evaluated later overwrites the name part evaluated first.

Configuring generation of a PDMS name

Requirement

- You have defined the structural behavior.
- You are familiar with the algorithm for generating the PDMS name. See also chapter Algorithm for generating a PDMS name (Page 109).

Elements of a PDMS name part

Each PDMS name part is defined by the following elements:

Row	Description
"Prefix"	Prefix of the PDMS name part. • Optionally: • Any number of characters
"Suffix"	The suffix of the PDMS name part. • Optionally: • Any number of characters
"Data item"	The data item of the PDMS name part. Permitted items: • Composed expression which returns the name or label of an object from the unit tree or location tree • String parameter
"Start index"	The point where a name part is inserted in the full PDMS name. If nothing is entered, the name part is appended to the name parts composed up to this point.
"Number of characters"	The number of characters of the data item that are applied for the name part. • Number • Any number of characters permitted

Procedure

1. Open the properties of the subclass definition object of the "Cable" or "Tagged item" class.
2. Select the "Attributes > Name mapping" tab.
3. Identify where objects of the subclass are located or need to be created in your COMOS object structure. Split the overall path into its individual nodes.
4. Analyze how the PDMS name for objects of the subclass is to be structured. You decide how many name parts and therefore columns you are using.

5. Enter a data item in at least one column.
   The data item of a mapping entry contains either COMOS data or PDMS data. The following data is permitted:

COMOS data	PDMS data
• Return value of a String parameter which returns COMOS data.	Return value of a String parameter which returns PDMS data.
• An expression which returns the name or label of an object in the unit tree.	/
• An expression which returns the name or label of an object in the location tree.	/

   Notation of the data item:

Notation of the String parameter	Notation of the composed expression
"!<Name of a local String parameter>" example: "!Area"	First element: • In unit mapping: "U" • In location mapping: "L"
"!!<Name of a global String parameter>"	Second element: • "<Number>": Structure level in the tree on which the object is located In unit mapping: Starting from the project root In location mapping: Starting from the object "Folders for class/ subclass definitions (PDMS)"
	Third element: • "N" or "L": Use the name or label of the object
	Example: "U2L": Use the label of an object on the second level underneath the project root in the unit tree.

6. Enter a start index and a number of characters in the same column.
7. Optional: Enter a prefix and a postfix.
8. Check which COMOS nodes and PDMS name parts can be mapped directly to one another. Configure a shared column for these nodes and name parts.
9. Check which COMOS node does not have a corresponding PDMS name part and vice versa.
   Configure a separate column for each of these nodes and name parts.
   Examples of COMOS nodes which do not have a corresponding PDMS name part:
   – Folder
   – The PDMS root object in the "Locations" tab
   – Other objects that are only used to structure the COMOS data
10. Configure the columns as described in the table below.

Algorithm for generating a COMOS object path

To find the COMOS object to which a PDMS/E3D object is to be connected, the "Name mapping" tab is evaluated at the used subclasses.

- To create an object in COMOS and complete its owner structure, the base objects that are to be used must be known for each hierarchy level.
- To create the owner structure, COMOS evaluates the "Base object" or "Base object from structure" row.
- To create the object which is actually searched for, the interface evaluates the UDS ":ComosBaseOb" of the PDMS object.
  If the UDA is empty, the interface evaluates the "Base object for creating objects" control group in the "General" tab of the subclass definition object.
  If no base objects are specified in the "Base object for creating objects" control group, the interface evaluates the "Base object" or "Base object from structure" row in name mapping.

This documentation assumes that this setting is made in the "General" tab.

Algorithm

1. The interface checks the subclass of the PDMS/E3D object.
2. The interface evaluates the creation mode of the corresponding subclass definition object: "General" tab, "Structural behavior" control group, "Creation mode" list:
   - "Create in unit tree": Unit mapping is performed.
   - "Create in location tree": Location mapping is performed.

The execution of the mapping tables involves the following steps:

1. If the list entry "Yes" is selected in the "Base object from structure" row:
   - The interface assumes that you are working in a KKS structure.
   - COMOS navigates to the project structure level that is specified in the current column.
   - In a project structure, several objects with different base objects can be prepared for a single level. The interface compares the data item or fixed name of the column with the name mask of the possible base objects.
   - The object is created with the base object whose name mask matches the data item or fixed name.
   - The data item or fixed name is entered in the "Name" property of the new object.

   If the list entry "No" is selected in the "Base object from structure" row:
   COMOS navigates to the base object that is specified in the "Base object" row of the current column.
2. The base object that is to be used for creating the current hierarchy level is derived from the "Base object" row.
3. The name of the COMOS object to be created is derived from the "Fixname" row.
   The entire COMOS hierarchy of the object is composed according to this principle.

Result

- The string written to the path list produces the path to the COMOS interface object you are looking for.
- If the object does not exist yet, the mapping table returns all needed information in order to create it. The name part of the last mapping entry is written to the "Name" property of the COMOS object.
- If the interface detects that the owner structure of the object you are looking for is incomplete: The mapping table returns all information required to complete the owner structure.

Configuring the generation of a COMOS object path

The next chapter describes the generation of a COMOS object path.

Requirement

- You have defined the structural behavior.
- You are familiar with the algorithm for generating the COMOS object path. See also chapter Algorithm for generating a COMOS object path (Page 113).

Elements for generating a COMOS path

Row	Function	Description
"Base object"	Base object used to create an object.	Used to complete the owner structure of the COMOS object or to create the object in COMOS. You only use one of the two rows.
"Base object from structure"	This row is only used for KKS structures. Specifies whether the base object from the KKS base data structure is to be used. • "Yes": If the project is linked to a project structure, the project structure is used for the generation of the COMOS objects. • "No": Even if a project structure is available and linked, it is not used.	
"Fixname"	Name of a COMOS object within the path to a COMOS object.	

Procedure

1. Open the properties of the subclass definition object of the "Cable" or "Tagged item" class.
2. Select the "Attributes > Name mapping" tab.
3. To define the base object of the first hierarchy level, enter the following in column "1":
4. When you are working in a KKS structure: In the "Base object from structure" row, select whether the project structure is to be used when generating the COMOS objects.

5. If you have selected "No" or are not working in a KKS structure: Enter which base object is to be used for generating the object.
   Possible values:
   - SystemFullName of a base object
     Use "\" to separate the individual nodes. Without quotation marks
   - Local String parameter: "!<Name of a local String parameter>"
   - Global String parameter: "!!<Name of a global String parameter>"
6. To define a name for the object that is to be created, enter a value in the "Fixname" column.
   Possible values:
   - Local String parameter: "!<Name of a local String parameter>"
   - Global String parameter: "!!<Name of a global String parameter>"
   - Fixed string: Without quotation marks
7. Repeat the steps for the remaining hierarchy levels.
   The column that is used last represents the object that is actually going to be created.
   This documentation assumes that the base object is defined in the "Base objects for creating objects" control group in the "General" tab of the subclass definition object. This means you do not define a base object in the mapping tables in the column that was used last.

2.6.12 Creating and configuring query subclasses

2.6.12.1 Workflow

Users who wish to execute and view COMOS queries in PDMS first have to complete the following preparatory steps:

- Create however many COMOS queries the user is to execute in PDMS.
  These queries are called main queries in the interface.
- Configure the main queries in the usual COMOS way.
- Create a class definition object for the "Query" class.
- For each main query: Create a subclass definition object.
- Configure the subclass definition objects.
- If the user should have the option to select the start object of a main query from a list of objects in PDMS:
  - Create a COMOS query whose result list returns the possible start objects
    This query is called the start object query in the interface.
  - Configure the start object query.
  - Configure the access to the start object query for the subclass definition object.

2.6.12.2 Creating and configuring query subclasses

Procedure

1. For each query that is started from PDMS and viewed, create a main query.
2. Configure the main query.
   If you are using an already existing query as the main query, this step is not necessary.
3. If the user should have the option to select the start object of a main query from a list of objects in PDMS:
   Create a start object query and configure it in such a way that it returns useful start objects for the main query.
   If you are using an already existing query as the start object query, this step is not necessary.

2.6.12.3 Creating class and subclass definitions for queries

Creating class for queries

1. Click the "Units" tab in the Navigator.
2. Select the "Folders for class/subclass definitions" object.
3. Select "New > Class - queries (PDMS)" from the context menu.

Creating query subclass definitions

To create a subclass definition object you use to manage a main query and its start object query, proceed as follows:

1. Select the object "Class - queries (PDMS)" in the Navigator.
2. Select the command "New > Query subclass definition" from the context menu.
   A subclass definition for queries is created.
3. Open the "Attributes > General" tab in the properties of the subclass definition.
   "PDMS settings" control group:
   – "PDMS name" field:
     Enter a name for the subclass.
   – "Description" field:
     Enter a description. If the user has selected the query in PDMS, the description is displayed in PDMS: "Query Comos" window, "Query" tab, "Description" field

2.6.12.4 Configuring a subclass definition object for queries

Procedure

To configure the subclass definition object you use to manage the main query and the start object query, proceed as follows:

1. Open the properties of the subclass definition object.
   See also chapter Creating class and subclass definitions for queries (Page 116).
2. Switch to the "Attributes > General" tab.
3. In the "Output format" control group, select the format in which the result list of the query is transferred to PDMS.
   – CSV
   – XML
4. If CSV is set as the output format, define the column separator in the "Separator" field. You can define any character as separator.
5. Drag&drop the main query from the Navigator into the "Main query" control group, "Query object" field.
6. Drag&drop an object into the "Start object" field.
   The name of the referenced start object is displayed to the user in PDMS: "Comos Query" window, "Query" tab, "Start object" field
   If you are using an already existing query as the main query, you can define a different start object than the one set in the main query.
7. Drag&drop the base objects according to which the query is to filter the result list from the Navigator to the "Base objects" column.
   If you are using an already existing query as the main query, you can have the query filtered in PDMS by base objects other than those defined by the main query in COMOS.
8. "Query for start objects" control group:
9. If the user is to have the option of selecting the start object of a main query from a list of objects in PDMS, configure the "Query for start objects" control group.
   – Drag&drop the query for start objects from the Navigator into the "Query object" field. The start object query returns the list of possible start objects in PDMS.
   – Drag&drop an object into the "Start object" field.
   – Drag&drop the base objects according to which the query is to filter the result list from the Navigator to the "Base objects" column.
     If you are using an already existing query as the start object query: You can filter the query in PDMS on the basis of base objects other than those defined by the main query in COMOS.

Evaluation priority for the start object

You can define the start object of the query at various places. The following table lists these places.

If a start object has been defined in multiple places, the places which appear further down the table overwrite those above them.

Start object of the main query	Start object of the query for start objects
Start object that is set in the COMOS query	Start object that is set in the COMOS query
Start object that was set for the subclass definition object	Start object that was set for the subclass definition object
Start object that the user defined in PDMS: "Comos Query" window, "Execute" tab	/

2.6.13 Configuring "Import DocLinks"

2.6.13.1 Defining the entry for the target document in PDMS

The "Import DocLinks" operation is called in PDMS. It creates DocObjs for draft objects under a COMOS document specified in the draft structure.

To import draft objects into COMOS as DocObjs, the COMOS document for which the DocObjs are being created must be known.

For this purpose, you should define an attribute or a UDA in which the user will subsequently enter the name or the SystemUID of the target document.

You decide the levels of the draft structure at which it is possible to specify target documents.

Defining target documents on different levels of the draft structure

You can specify a target document at more than one level of the draft structure. This enables the user to start the DocLink import as required at different levels of the draft structure, e.g. for all draft objects of a region, for an individual view or for an individual drawing.

To do this, repeat the below described procedure for all levels for which the "Import DocLinks" operation needs to be available.

Procedure

To define where the target document is specified, proceed as follows:

1. Define whether the user enters the name of the target document in an attribute or in a UDA and which attribute or UDA is used.
2. Define the level of the draft structure on which the user specifies the name of the target document.
3. If you are using a UDA, create the UDA.

2.6.13.2 Configuring "Import DocLinks" operation in COMOS

Procedure

1. Create a "Document subclass definition" object.
2. Define a String parameter in the name directory of the subclass definition object. The String parameter is to access the attribute or UDA in which the name or SystemUID of the target document is entered. See also chapter Defining the String Parameter for the document name (Page 119).
3. Configure the general properties of the subclass definition object. See also chapter Configuring general settings (Page 119).
4. Optional: On the "Owner restriction rules" tab, specify whether the owners of the template objects for which DocObjs are being created have to meet certain conditions.

2.6.13.3 Defining the String Parameter for the document name

Define a String parameter which, from PDMS, reads out the name of the COMOS document under which the DocObjs are imported.

Procedure

1. Open the properties of the "Document subclass definition" object and switch to the "Name directory" tab.
2. Configure the "Name directory" table:
   - "Name" column: Enter the name of the String parameter.
   - "PDMS attribute/expression" or "GetFunction" column: Access the value of the attribute or the UDA in which the document name or the SystemUID is saved in PDMS. See also chapter Working with String Parameters in the name directory (Page 93).
   - "PDMS type" column: Enter "String".

2.6.13.4 Configuring general settings

Procedure

To define the general settings of the "Document subclass definition" object, proceed as follows:

1. Open the properties of the "Document subclass definition" object.
2. Switch to the "Attributes > General" tab.
3. "Document ID" field:
   - Enter the name of the String parameter which reads out the document name from PDMS.
   - Do not precede the name of the String parameter with a "!" character.

4. Optional: Define filters in the "Base object for document search" control group:
   During the search, the interface restricts itself to documents which meet the criteria defined here. This speeds up the search.
   The following filters are possible:
   – "Document type" list: Define the document type of the document.
   – "Base object for document search" field: Define the base object the document uses.
   – "Base object" field: Define the node below which the document is searched.
5. Optional: Activate the "Filter unnamed objects" option:
   In PDMS, only objects with a name are displayed in the result list in the "Export to Comos" window.
6. Enter the PDMS element types of the following template objects in the table: Template objects with a DDNM attribute that references a design object that is connected to a COMOS object.

2.6.13.5 Defining owner restriction rules for draft objects

Procedure

You can define owner restriction rules which the template objects have to meet to be imported as DocObjs.

For this purpose, proceed in the same way as with the definition of owner restriction rules on subclass definition objects of the "Cable" and "Tagged item" classes. See also chapter Specifying restriction rules for owners (Page 89).

2.6.14 Synchronizing classes and subclasses in COMOS and PDMS

Initial situation

To ensure that COMOS and PDMS use the same settings, the interface does not directly access the settings for the "Folders for class/subclass definitions" object and for the class and subclass definition objects. Instead, the settings are written to an XML file in the exchange directory. COMOS and PDMS load the settings from the file to the working memory of your computer.

When an interface operation is called, the current version number of the "Folders for class/subclass definitions" object is transferred. The operation checks whether the number is identical to the version number of the settings stored in the work memory.

- Yes: The settings stored in the work memory are used.
- No: The file is loaded to the work memory again. The settings are synchronized.

If synchronization was successful, ensure that the file in the exchange directory contains the current settings.

Update the file whenever you have performed one of the following actions:

- You changed the settings of the "Folders for class/subclass definitions" object.
- You created a new class or subclass definition object.
- You changed the settings of a subclass definition object.

Procedure

To update the file in which the settings for the "Folders for class/subclass definitions" object and the class and subclass definition objects are saved, proceed as follows:

1. Select the "Folders for class/subclass definitions" object in the Navigator.
2. Select the following command from the context menu: "PDMS > Save subclasses".

Result

- The version number at the "Folders for class/subclass definitions" object is incremented.
- The standard table for subclasses is updated.
- The settings for the "Folders for class/subclass definitions" object and for the class and subclass definition objects are written to the file in the exchange directory.

2.6.15 Importing classes, subclasses and "Folders for classes/subclasses" object from XML file

If you are working in different project structures with different requirements, the classes, subclasses and the "Folders for classes/subclasses" object must be configured according to the respective requirements. To avoid having to edit the objects every time you switch to a new project structure, you can import the completely configured objects from an XML file and have them created in COMOS.

This means you can create an XML file for each project structure and import it as needed.

Requirement

You have access to at least one XML file which was previously created or updated using the "Save subclass" function. You can name the file as you wish.

The XML files are located in the directory that is specified in the `SubclassesFile` node in the initialization file.

Procedure

1. Select the "Folders for class/subclass definitions" object in the Navigator.
2. Select the following command from the context menu: "PDMS > Import subclasses".
3. Select the following command from the context menu: "PDMS > Save subclasses".

Result

- If a "Folders for classes/subclasses" object already exists, the object is deleted with all its classes and subclasses.
- The "Folders for classes/subclasses" object from the XML file is created in the Navigator together with its classes and subclasses.

2.6.16 Importing AVEVA design templates

Initial situation

You can assign an AVEVA design template to an interface object as a model.

To do this, you need to import the design templates from PDMS into COMOS.

Procedure

1. Switch to the "Base objects" tab in the Navigator.
2. Select the following node: "@20 > D30 > A30 > Y00R00038 Objects for design template structure".
3. In the context menu, select the following command: "PDMS > Update design templates".

Result

- The GTypes of all templates are evaluated in PDMS.
- The templates are imported from PDMS to COMOS. During this process, the interface proceeds as described below.

Managing the design templates in the base data

The design templates are managed under the following node in the base data:

"@20 > D30 > A30 > Y00R00038 Objects for design template structure"

The design templates are managed sorted by GType:

- Nodes which represent the GTypes of the templates are located on the first level below the "Objects for design template structure" object.
- The nodes for the templates are located on the second level. A node is created for each template.

Algorithm for importing the design templates into the base data

- For each GType found in PDMS, a search is performed below the "Objects for design template structure" object for a node whose name matches the name of the GType.
- If a node of this type does not exist yet, it is created. The name of the GType is entered in the "Name" property of the node.

- A node for the design template is created underneath the GType node.
- If the design template already exists, it is updated. No data is deleted during the update operation.
  Example:
  - A sub template was deleted in PDMS.
  - The sub template remains available in COMOS even after the update.
- The attributes of the template node are automatically set when the template node is created or during the update.

2.6.17 Configuring COMOS interface objects

2.6.17.1 Assigning the class, subclass, and PDMS element type

Procedure

To assign a class, a subclass, and PDMS element types to an interface object, proceed as follows:

1. Click on the "Base objects" tab in the Navigator.
2. Navigate to an interface base object.
3. Open the properties of the interface base object.
4. Switch to the "Attributes > External 3D Interface" tab.
5. Configure the tab as described below.
6. "Class" list:
   - Select a class.
   - Do not select the "Undef" value. Otherwise, the object will not be recognized as an interface object.
7. "Subclass" list:
   Select a subclass. Only subclasses permitted for the selected class are offered.
8. "PDMS element type" list:
   Select a PDMS element type. Only those element types entered for the selected subclass are offered.

Overwriting the preselection in the engineering view

In the database, the edit mode of these attributes is configured in such a way that users can overwrite the inherited settings in the engineering view, if this is necessary.

See also

Attributes of the "External 3D Interface" tab (Page 138)

2.6.17.2 Overwriting the inherited model information

Requirements

- The following has already been assigned to the interface object:
  - Class
  - Subclass
  - PDMS element type
- AVEVA templates have been imported.

Inheritance of subclasses

The database is configured in such a way that the interface objects apply the model information settings from their subclass. For this purpose, the model attributes of the interface objects are linked to those of the subclass definition objects.

Link type: Static link via the "GetLinkedSpecification" script function.

You can overwrite the settings for the base objects of the interface objects taken from the subclass.

Procedure

To define the model information at the base object of the interface object, proceed as follows:

1. Click on the "Base objects" tab in the Navigator.
2. Navigate to an interface base object.
3. Open the properties of the interface base object and switch to the "Attributes > External 3D Interface" tab.

4. Define how the model will be generated in the "Model" control group, "Mode" list. Select one of the following modes:

Mode	Interface object has subclass of "Cable" class	Interface object has subclass of "Tagged item" class	Result
"Design template"	/	Available if the subclass has the "EQUI" PDMS element type	The geometry is based on a design template.
"PDMS element"	/	Available	The geometry is copied from an existing PDMS element.
"Function"	Available	Available	The geometry is generated by a function which is also performs other initializations.
"None"	Available	Available	A box is generated.

5. Depending on which mode you have selected, you now need to define the details for the generation of the model:
   - "Function" mode:
     Enter the function call in the "Function" field.
   - "PDMS element" mode:
     Enter the name of the PDMS element whose geometry is being copied in the "Element" field.
   - "Design template" mode:
     Define the main template in the "Design template" control group, along with the used sub templates which, if applicable.

Defining design templates

If the following conditions are met, you can select design templates as the basis for your model:

- The subclass of the interface object has the "Tagged item" class.
- The subclass has the "EQUI" PDMS element type.

If you select a design template as the basis for your model, you need to configure the "Design template" control group.

Proceed as follows:

1. Select the main document template in the "Main document template" list.
   Only those templates whose GType has been entered in the subclass in the "GTypes for filtering" field are offered for selection.
   If the the "EQUI" PDMS element type is missing at the subclass, the list remains empty.
2. You can define up to five templates for secondary equipment in the "Sub template 1" to "Sub template 5" lists:
   Which templates are offered depends on the main template you selected.

Design templates in the engineering view

When COMOS creates an interface object in the engineering view, it automatically checks whether the interface object uses a design template. If it does, COMOS creates the main document template underneath the interface object and the sub templates underneath that.

COMOS creates the template objects when you assign a design template to an existing interface object.

- Name of the template object: "Y00R00038"
- Description of the template object: As defined at the base object of the template object

You can change the design parameters of the template objects in the engineering view: Properties of the template object, "Attributes > External 3D Interface" tab, attributes "Y00A02118AA01 DP 01" through "Y00A02118AA10 DP 10".

Overwriting the base object settings in the engineering view

Users can overwrite both the settings inherited from the subclass and the settings predefined at the base object in the engineering view.

Restoring the default settings

The settings inherited from the subclass can be restored in the engineering view using the "Refresh static links on tab" context menu.

See also

Attributes of the "External 3D Interface" tab (Page 138)

COMOS 3D Viewing

3

You can find documentation on this topic in the Installation Package of the "COMOS Walkinside" software. You can find additional information on this topic in the "COMOS Walkinside" manual, keyword "COMOS Walkinside Integration".

References 4

4.1 Category "Process engineering > PDMS/E3D Engineering Interface"

The category defines the following project parameters:

Description	Purpose
"Initialization file" field	Directory containing the initialization file.
"Exchange directory" field	These fields cannot be edited. Their content is taken from the configuration file.
"Communication directory" field	
"Document path" field	
"Configuration path" field	The configuration path cannot be edited. It is taken from the configuration file.
"Interface active" option	Button to activate COMOS PDMS Interface.

4.2 COMOS PDMS Integration

4.2.1 Structure of the standard table for PDMS element types

Structure of the standard table "Standard tables > @40 > Y00 > A10 > C10 > Y00N00465 PDMS element types"

Column name	Function
"Name"	• Index number • Used internally • Starts with 1
"Description"	Is displayed as a value in the interface for attributes to which the standard table has been assigned.
"Value 1"	The PDMS element type Notation as in PDMS
"Value 2"	PDMS module in which the PDMS element type is used. Possible values: • "Design" • "Draft"

See also

Maintaining the standard table for PDMS element types (Page 76)

4.2.2 Structure of the standard table for classes

Path in the database: "Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A01 Classes"

Column name	Function
"Name"	• Index number • Used internally • Starts with 1
"Description"	Is displayed as a value in the interface for attributes to which the standard table has been assigned.
"Value 1"	• Class name • Used system-internally
"Value 2"	Letter code: The letter entered here is used in the standard table for subclasses to assign a class to a subclass.

See also

Standard tables for classes and subclasses (Page 77)

4.2.3 Structure of the standard table for subclasses

Path in the database: "Standard tables > Y10 > M27 > A10 > Y10M27N00002 > Y10M27N00002A02 Subclasses"

Column name	Function
"Name"	• Index number • Used internally • Starts with 1
"Description"	Is displayed as a value in the interface for attributes to which the standard table has been assigned.
"Value 1"	• Subclass name • Used system-internally
"Value 2"	Letter code which assigns a class to the subclass. Only letters entered in the class standard table in the "Value 2" column are permitted.

See also

Standard tables for classes and subclasses (Page 77)

4.2.4 Attributes of the "Folders for class/subclass definitions" object

4.2.4.1 "General" tab

Content

The tab contains the following attributes:

Control element	Description
"Use zone and site mapping" option	Activates zone and site mapping.
"Feedback mode" option	Activated: The two applications can access and write to the data bidirectionally. Deactivated: Only COMOS can access and write to data from Walkinside.
"Subclass for pipe" list	Application case: COMOS works with a two-stage KKS structure which uses pipe sections and segments. Enter the subclass which is used to create or find the pipes in PMS under which the branches are located. Configure the here entered subclass as follows: • Creation mode and assign mode: "None" or • Creation mode: "Create in location tree" and Assign mode of the subclass for pipe sections: "Assign to object in location tree" and Location mapping of the subclass for pipe sections: Addresses objects of the subclass entered in the "Subclass for pipes" subclass in the object tree
"Folder for project scope query" field	Drag&drop the folder containing the queries for the project scope into this field.
"Operation messages" table	• "Operation" column Interface operations for which operation messages are available • "Show information" column Activates and deactivates the process logs.

See also

Synchronizing classes and subclasses in COMOS and PDMS (Page 120)

Restricting COMOS objects for the COMOS PDMS/E3D Engineering Interface (Page 54)

4.2.4.2 "Name directory" tab

Content

The tab contains the following table:

Table	Description
"Name directory"	Each line in the table defines a String parameter. The String parameters define variables which are globally available in the interface operations area. You configure an assignment between COMOS attributes and PDMS attributes in the subclass definition objects.

See also

Defining global variables (Page 79)

4.2.4.3 "Pre/post executables" tab

Content

The tab contains the following tables:

Table	Description
"Pre executable"	Registers the PML functions which are called before an interface operation is executed.
"Post executable"	Registers the PML functions which are called after an interface operation has been executed.

You can find information on the structure of the table and the configuration of the tab in chapter Using pre/post executables (Page 79).

4.2.4.4 "Character mapping" tab

Content

The tab contains the following table:

Table	Description
/	Defines which characters are permitted in COMOS but not in PDMS and are replaced during name mapping.

You can find information on the structure of the table and the configuration of the tab in chapter Using character mapping (Page 81).

4.2.4.5 "Unit mapping" tab

If a different unit has been assigned to an attribute in COMOS than in PDMS, the table defines how units in PDMS and COMOS are mapped to one another.

You can find information on the structure of the table and the configuration of the tab in chapter Using unit mapping (Page 81).

4.2.4.6 "Site mapping" and "Zone mapping" tabs

Content

The tabs contain the following tables:

Table	Description of the table
Table for site mapping	Defines how the SITE is found in PDMS for a COMOS object under which the object may be located.
Table for zone mapping	Defines how the ZONE is found in PDMS for a COMOS object under which the object may be located.

You can find information on the structure of the tables and the configuration of the tabs in chapter Using ZONE mapping and SITE mapping (Page 82).

4.2.5 Attributes of the subclass definition objects of classes "Cable" and "Tagged item"

4.2.5.1 "General" tab

"Base object for creating objects" control group

Field	Description
"Create in unit tree"	Master object, e.g. which is to be used for the import from PDMS to generate objects of this subclass in COMOS.
"Create in location tree"	Master object, e.g. which is to be used for the import from PDMS to generate objects of this subclass in COMOS.

See also chapter Defining the base object for the creation of interface objects (Page 87).

"Structural behavior" control group

Control element	Description
"Creation mode" list	This is where you define whether the COMOS object connected to the PDMS interface object is located in the unit tree or in the location tree. See also chapter Defining structural behavior (Page 86).
"Assign mode" list	This is where you define whether the COMOS interface object references another COMOS object and if the referenced object is located in the unit tree or in the location tree. See also chapter Defining structural behavior (Page 86).
Table	"PDMS element type" column: List with the PDMS element types which are permitted for the draft objects for which DocLinks are created. "Name" column: Types of PDMS objects to which the subclass is applied. See also chapter Defining PDMS element types (Page 85).

See also

Algorithm 2 for generating a COMOS object path (Page 107)

4.2.5.2 "Name mapping" tab

Content

The tab contains the following tables:

Table	Purpose
"Site mapping"	Table which defines site mapping.
"Location mapping"	Table which defines location mapping.
"Level rules"	Table which defines deviations from the standard procedures for site mapping and location mapping.

Information on the structure and configuration of the tables:

- Chapter Name mapping (Page 63)
- Chapter Configuring level rules (Page 109)
- Chapter Configuring site mapping and location mapping in name mapping (Page 100)

4.2.5.3 "Owner restriction rules" tab

Content

The tab contains the following attributes:

Control element	Purpose
"Element type of owner" list	List in which you define the element type of the owner of the connected PDMS object.
Table 1	Table in which you define expressions which have to be evaluated to "True" at the owner.
Table 2	Table in which you define filters which check whether a given owner attribute contains a specific occur string.

You can find information on the structure of both tables and the configuration of the tab in section Specifying restriction rules for owners (Page 89).

4.2.5.4 "Model" tab

Content

The tab contains the following attributes:

Control element	Purpose
"Mode" list	You specify the model type here.
"Function" field	Function call for generating the model.
"Element" field	Name of the PDMS element whose model is copied.
"GTypes for filtering"	GTypes of the design templates offered for this subclass. Separator: ";"

"Design templates" control group

List	Description
Main document template	List from which you select a design template for the main equipment. The entries in the list are filtered based on the entries in "TMPFILTER". The list is empty if the entry "EQUI" is missing in "TMPFILTER".
"Sub template"	Sub templates are design templates for the secondary equipment. The entries in the list are determined by the main template set in "PDMS004".

See also

Configuring the model (Page 91)

4.2.5.5 "Name directory" tab

Content

The tab contains the following table:

Description of the table	Purpose
"Name directory"	Definition of String parameters

You can find information on the structure of the table and the configuration of the tab in chapter Working with String Parameters in the name directory (Page 93).

4.2.6 Attributes of the subclass definition objects of the "Query" class

4.2.6.1 The "General" tab

"PDMS settings" control group

Control element	Description
"PDMS name" field	Name which users will see when the query is offered for execution in PDMS.
"Description" field	Description with which the query is displayed in PDMS once the user has selected it for execution.

"Output format" control group

Control element	Description
"Output format" list	Output format in which the result list of the query will be transferred to PDMS.
"Separator" field	Use a separator only If the output format is "CSV". The separator can be chosen at will.

"Main query" control group

Control element	Description
"Query object" field	Reference to the query object whose content is to be displayed in PDMS.
"Start object" field	Optional: Reference to a start object. Overwrites the start object set in the query referenced in the "Name" field.
"Base objects" table	Optional: References to the base objects by which the query filters the result list. You can set other base objects than those set in the query referenced in the "Name" field.

"Query for start objects" control group

Control element	Description
"Query object" field	Reference to the query returned by the list of possible start objects for the main query in PDMS.
"Start object" field	Optional: Reference to a start object for the start object query. Overwrites the start object set in the query referenced in the "Name" field.
"Base objects" table	Optional: References to the base objects according to which the query filters the result list. See "BaseObjectsMQ".

See also

Configuring a subclass definition object for queries (Page 117)

4.2.7 Attributes of the subclass definition objects of the "Document" class

4.2.7.1 "General" tab

Content

Control element	Description
"Document ID" field	Name of the string parameter which reads out from PDMS the name of the COMOS document under which the DocLinks are imported.
"Document type" list	The document types are used as filters to speed up the search for the COMOS document.

Control element	Description
"Base object for document search" field	Reference to the base object of the COMOS document. The base object is used as a filter to speed up the search for the COMOS document.
"Base object" field	Reference to the start object under which the search for the COMOS document is carried out. The base object is used as a filter to speed up the search for the COMOS document.
"Filter unnamed objects" option	Option for filtering the result list of the "Export to Comos" window. Activated: The result list contains only objects with names.
Table	"PDMS element type" column: List with the PDMS element types which are permitted for the draft objects for which DocLinks are created. "Name" column: Types of PDMS objects to which the subclass is applied. See also chapter Defining PDMS element types (Page 85).

4.2.7.2 "Owner restriction rules" tab

See also chapter "Owner restriction rules" tab (Page 135).

4.2.7.3 "Name directory" tab

See also chapter "Name directory" tab (Page 136).

4.2.8 Attributes of the interface objects

4.2.8.1 Attributes of the "External 3D Interface" tab

Function

COMOS objects require the "External 3D Interface" tab in order to be used as an interface object.

The COMOS PDMS Interface evaluates this tab whenever it processes interface objects. Without this tab, the object is not processed.

Database

In the database, tabs have been prepared for the following interface objects:

- Objects of "Tagged item" class
- Objects of "Cable" class
- For nozzles

The tabs for these objects are located in the following catalog:

"@40 > A20 > Y00 > A10 > A10 > Y00T00012 > A03 > B01 Tabs PI-objects"

From there, they are passed on to the PID device catalog in the database.

System-internal attributes

The following attributes are used system-internally. Do not configure or delete them:

- "ComosCheckUID Check SystemUID"
- "PDMSNAME Name of the corresponding object in PDMS"
- "PDMSRefNo Reference number"

Remaining attributes

Define the following properties of the interface object on the "External 3D Interface" tab:

- The class and subclass
- The PDMS element type
- The model of the object

Details regarding the corresponding attributes and their configuration: See the links below.

See also

Assigning the class, subclass, and PDMS element type (Page 123)

Overwriting the inherited model information (Page 124)

4.2.9 Attributes of the design template objects

4.2.9.1 Attributes of the "External 3D Interface" tab

Content

The tab contains the following attributes:

Field	Description
"PDMS GTYPE"	GType of the design template
"Purpose"	Purpose of the design template

"Design parameter" control group

Field	Description
"DP 01" to "DP 10"	Field in which the value of the corresponding design parameter is entered. Can be modified by the user in the engineering data.
/	Field for the description of the design parameter from PDMS.

The following attributes are dynamically created while the template object is being created in the engineering data:

Field	Description
/	Is only generated if one or more design templates for secondary equipment were specified for an interface object: • "<x>": Defines which of the five possible sub templates the current sub template belongs to. • "<y>": Several templates are available for selection for each item of secondary equipment. "<y>" defines which template the attribute refers to. • Value: The Catref of the sub template selected at the interface object.